密级:_____

中国科学院大学
University of Chinese Academy of Sciences

硕士学位论文

MIPS64 平台下 JVM 性能优化研究与实现

作者姓名：王一光

指导教师：章立生　正高级工程师

中国科学院计算技术研究所

学位类别：工学硕士

学科专业：计算机系统结构

研究所：中国科学院计算技术研究所

2015 年　5 月

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

Research and Implementation about Performance Optimization of JVM on MIPS64 platform

By

Wang Yiguang

A Dissertation/Thesis Submitted to
University of Chinese Academy of Sciences
In partial fulfillment of the requirement
For the degree of
Master of Computer Architecture

Institute of Computing Technology Chinese Academy o f Sciences
May, 2015

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

声　　明

我声明本论文是我本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，本论文中不包含其他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：　　日期：

论文版权使用授权书

本人授权中国科学院计算技术研究所可以保留并向国家有关部门或机构送交本论文的复印件和电子文档，允许本论文被查阅和借阅，可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编本论文。

（保密论文在解密后适用本授权书。）

作者签名：　　导师签名：　　日期：

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

摘 要

Java 应用具有良好的跨平台特性，广泛应用于服务器领域。Java 虚拟机作为 Java 程序赖以运行的基础平台，对 Java 应用执行效率的影响至关重要。随着虚拟机由 32 位向 64 位过渡，服务器软件功能与复杂度不断增加，对 Java 应用执行性能需求日益提升，在精简指令体系结构平台，如 MIPS 下虚拟机性能遇到新的瓶颈。对 MIPS64 平台下的 Java 虚拟机性能优化方法的研究具有重要意义。

本文完成的主要工作和贡献如下：

第一，针对 MIPS64 系统下虚拟机因指针膨胀而产生的性能瓶颈进行分析，提出并实现一种通过在虚拟机内存中增设全局立即数区缓存 Java 方法入口地址的方案，将多条对方法入口地址的加载指令转化为一条访存指令，从而提升方法调用的执行效率，优化系统性能。

第二，以 SpecJVM 2008 为实际用例，对虚拟机运行时类库方法的执行时间和使用频度进行度量和统计，发现在一些典型场景下，对类库方法的调用具有时间局部性，存在少数使用频度较高的热点方法，占据绝大部分的执行时间分成，这部分高频度热点方法的实现具有潜在的优化空间。

第三，基于运行时类库使用频度的度量统计，提出了使用 JNI 调用机制和使用 Intrinics 机制优化热点方法的方案，通过将高频度热点方法进行本地化并内置于虚拟机中，提高了热点方法的执行效率。以 SpecJVM 2008 为目标用例，分别对两种优化方案进行应用，进一步提升系统性能。

在龙芯软硬件平台上对本文提出的优化方案进行了效果验证和测试。其中方法调用机制优化后平均性能提升 5%，对运行时类库热点类投入优化后平均性能提升 4.07%，最高提升 11.93%，本文优化成果已集成于龙芯 JDK 实际应用产品中。

关键词：Java 虚拟机，MIPS64，方法调用，运行时，性能优化

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

ABSTRACT

Java Applications are widely used by servers because of their platform independence. Java Virtual Machine, as the fundamental platform for running Java programs, is of key importance for the executing efficiency of Java applications. As the JVM develops from 32-bit to 64-bit, the complexity of functions keeps increasing, and the requirement of performance keeps growing as well. New bottleneck appears on RISC platforms such as MIPS, so it is urgent and meaningful to do research about new ways to optimize the performance of JVM on MIPS64 platforms.

The completed work and main contributions of this article are as follows:

Firstly, analyze the performance bottleneck of JVM caused by pointer expansion on MIPS64 platforms and propose a solution in which we add a new region called Global Immediate Table in RAM to cache the entrance addresses for Java methods. An instruction to access the memory is used during the process instead of using several instructions to load an entrance address into a register. With this solution, the executing efficiency is enhanced and the JVM performance is promoted.

Secondly, measure the frequency of Java methods in JDK libray and get statistical results, taking SpecJVM 2008 as the standard benchmark. We discover that in some typical scenarios, temporal locality appears when Java methods in JDK library executed. There are a few methods which are only a small part of JDK library but occupy most part of the run time. A potential opportunity exists in these Java methods.

Thirdly, a solution is proposed to optimize the methods frequently invoked, using JNI and Intrinsics mechanism provided by JVM. The executing efficiency of “hot” methods is promoted by making the methods native and intrinsic. The solution is applied to SpecJVM 2008 and further enhances the performance of JVM.

The solutions proposed by this article are verified and tested on the hardware and software platform of Loongson. The new solution to load entrance addresses for Java methods improves the performance of SpecJVM 2008 by 5% on average. The optimization for “hot” methods in JDK library improves the performance of SpecJVM 2008 by 4.07% on average, and the highest performance increasment reaches 11.93%. The results of the work have been integrated into the Loongson products.

Key Words: JVM, MIPS64, runtime, method invocation, performance optimization

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

目　录

摘　要 ……I

ABSTRACT ……III

目　录 ……V

图目录 ……VII

表目录 ……IX

第一章 引言 ……1

1.1 研究背景 ……1

1.2 Java 虚拟机介绍 ……2

1.2.1 Java 虚拟机的外部视图 ……2

1.2.2 Java 虚拟机的内部视图 ……3

1.3 MIPS64 体系结构介绍 ……4

1.4 本文主要工作 ……5

1.5 本文组织结构 ……5

第二章 Java 虚拟机性能优化研究 ……7

2.1 Java 虚拟机执行框架 ……7

2.2 解释器及其性能优化 ……8

2.2.1 解释器工作原理 ……8

2.2.2 解释器性能优化 ……9

2.3 即时编译器及其性能优化 ……10

2.3.1 即时编译器的工作原理 ……10

2.3.2 即时编译器的代码优化 ……12

2.3.3 即时编译器的运行时优化 ……13

2.4 垃圾回收机制 ……14

2.5 本章小结 ……15

第三章 64 位虚拟机方法调用优化 ……17

3.1 研究背景 ……17

3.1.1 32 位虚拟机与 64 位虚拟机对比 ……17

3.1.2 MIPS 体系结构 64 位立即数加载方式 ……19

3.1.3 64 位虚拟机中的立即数来源分析 ……19

3.2 原有方法调用机制 ……19

3.2.1 现有虚拟机方法调用及解析过程.....17
3.2.2 现有虚拟机方法调用机制性能分析.....20
3.3 系统设计.....21
3.3.1 静态数据结构.....21
3.3.2 动态运行控制流.....23
3.4 算法实现.....25
3.5 本章小结.....28
第四章 虚拟机运行时类库优化.....29
4.1 研究背景.....29
4.2 类库频度度量.....29
4.3 Intrinsics 机制优化.....31
4.3.1 Intrinsics 机制工作原理.....31
4.3.2 利用 Intrinsics 机制实现优化.....32
4.4 JNI 调用优化.....33
4.4.1 JNI 机制介绍.....33
4.4.2 利用 JNI 机制实现优化.....35
4.5 本章小结.....36
第五章 测试与验证.....39
5.1 测试与验证平台介绍.....39
5.1.1 龙芯处理器硬件平台介绍.....39
5.1.2 龙芯系统软件平台介绍.....39
5.1.3 SpecJVM 2008 测试集介绍.....40
5.2 64 位立即数加载优化测试.....41
5.2.1 立即数加载优化即时编译代码效果对比.....41
5.2.2 立即数加载优化性能对比测试.....43
5.3 运行时类库优化测试.....47
5.4 本章小结.....48
第六章 总结与展望.....51
6.1 本文工作总结.....51
6.2 未来研究方向.....52
参考文献.....53
致 谢.....i

图目录

图 1-1 服务器技术架构 1
图 1-2 Java 虚拟机外部视图 2
图 1-3 Java 虚拟机内部视图 3
图 2-1 Java 虚拟机全景图 7
图 2-2 线索化解释器与原始解释器对比 9
图 2-3 Java 虚拟机 JIT 编译器执行框架 11
图 2-4 Client 版即时编译器基本架构 12
图 2-5 三种垃圾收集算法对比 14
图 3-1 MIPS 体系结构 64 位立即数加载过程 18
图 3-2 MIPS64 立即数加载指令序列 18
图 3-3 现有 MIPS64 虚拟机方法调用解析过程 20
图 3-4 全局立即数表结构设计 21
图 3-5 方法入口地址查找方式 22
图 3-6 方法入口地址填入立即数表过程 23
图 3-7 方法首次被调用执行过程 24
图 3-8 方法后续被调用执行过程 25
图 4-1 虚拟机运行 xml.transform 执行模式时间分成 30
图 4-2 虚拟机运行 xml.transform 方法调用频度统计 30
图 4-3 Intrinsics 机制优化工作原理 31
图 4-4 反汇编 StrictMath.floor()方法结果 33
图 4-5 Java 本地接口工作结构 33
图 4-6 JNI 使用方法示例 34
图 5-1 实例 java.util.HashMap::put 方法翻译后代码 41
图 5-2 优化后 java.util.HashMap::put 方法翻译后代码 42
图 5-3 默认参数下立即数加载优化效果测试 46
图 5-4 使用 UseNUMA 参数立即数加载优化效果测试 46
图 5-5 运行时类库热点类优化效果测试 48

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

表目录

表 2-1 字节码分类及示例 8
表 2-2 解释器、静态编译器、即时编译器比较 16
表 3-1 JVM 规范提供的方法调用相关的字节码 19
表 3-2 虚拟机内部公共例程入口介绍 25
表 3-3 MIPS 寄存器约定及其用法 27
表 4-1 JNI 提供的类和对象的操作函数 35
表 4-2 JNI 提供的对域和方法的操作函数 35
表 4-3 Intrinsic 方法与 JNI 调用比较 36
表 5-1 测试验证平台配置介绍 39
表 5-2 SpecJVM 2008 测试集项目介绍 40
表 5-3 默认参数下立即数加载优化 SpecJVM 2008 测试分值 43
表 5-4 UseNUMA 参数下立即数加载优化 SpecJVM 2008 测试分值 44
表 5-5 运行时类库优化 SpecJVM 2008 测试分值 47

中国科学院计算技术研究所 鄢銮 2025-03-14 14:47:37

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

第一章　引言

1.1　研究背景

现代主流计算机网络架构一般采用客户端/服务器模式，由服务器接收来自不同客户的请求，完成处理操作并将结果返回给客户端。服务器上运行的大多数应用程序由客户端通过网络访问，称为 Web 应用(Web Application)。Web 应用的优势在于客户除浏览器外无需安装其他软件就能够直接访问使用。

一个 Web 应用通常由多个组件构成，称为 Web 组件(Web Components)，每种组件用于完成特定的功能，组件之间协同工作将 Web 服务展示给外界。实际应用中，Web 应用由动态资源和静态资源组成，动态资源主要为 Java Servlet 或 JSP 页面，静态资源主要是 HTML 或图像文件，这些组件协同工作构成了一整套服务。其中，Java Servlet，JSP 提供了 Web 应用的核心业务实现，这部分组件运行依托于 JSP 容器，实际应用场景中由中间件提供支持。主流中间件如 Tomcat、Jetty 采用纯 Java[1]语言编写，同大量 Java 应用一样运行在 Java 虚拟机[2]之上。Java 虚拟机同数据库一同依托于硬件平台与操作系统之上，为应用软件提供支持，从而形成一整套层次结构。

现代服务器典型技术结构如下：

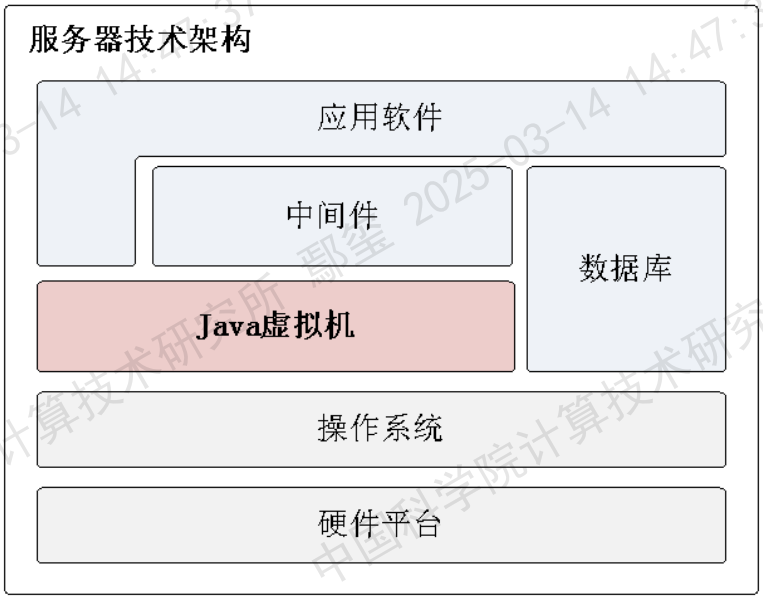

图 1-1 服务器技术架构

Java 是由 Sun Microsystems 公司推出的面向对象程序设计语言，其前身是

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

1991 年为消费类电子产品的嵌入式芯片设计的 Oak 语言，于 1995 年正式推出。Java 语言因其平台无关性和安全性，为 Web 应用提供了有力支持，因此迅速得到推广，已经成为移动互联网时代的主流开发语言。

Java 语言的跨平台特性和安全性，需要 Java 虚拟机提供的运行环境支持。Java 虚拟机对上层屏蔽了底层平台的实现细节，使得现代 Web 应用和主流中间件能够在不同平台上自如运行，因而在服务器技术体系中具有重要地位。

1.2 Java 虚拟机介绍

Java 不同于一般的静态编译型语言，Java 程序运行前首先需要被编译成一种称为字节码[3]的二进制表示形式，然后依赖各种不同平台上的 Java 虚拟机来解释执行这种字节码。Java 虚拟机在解释字节码时，会将其转换成具体平台上的本地机器指令序列执行。

Java 虚拟机作为 Java 语言底层实现的基础平台，屏蔽了与具体体系结构相关的信息，使得 Java 应用在不同硬件平台上运行时无需重新编译。Java 源程序只需生成在 Java 虚拟机上运行的字节码文件，就可以在多种平台上不加修改地运行，即“一次编译，到处运行”。

1.2.1 Java 虚拟机的外部视图

Java 虚拟机的外部视图如下所示：

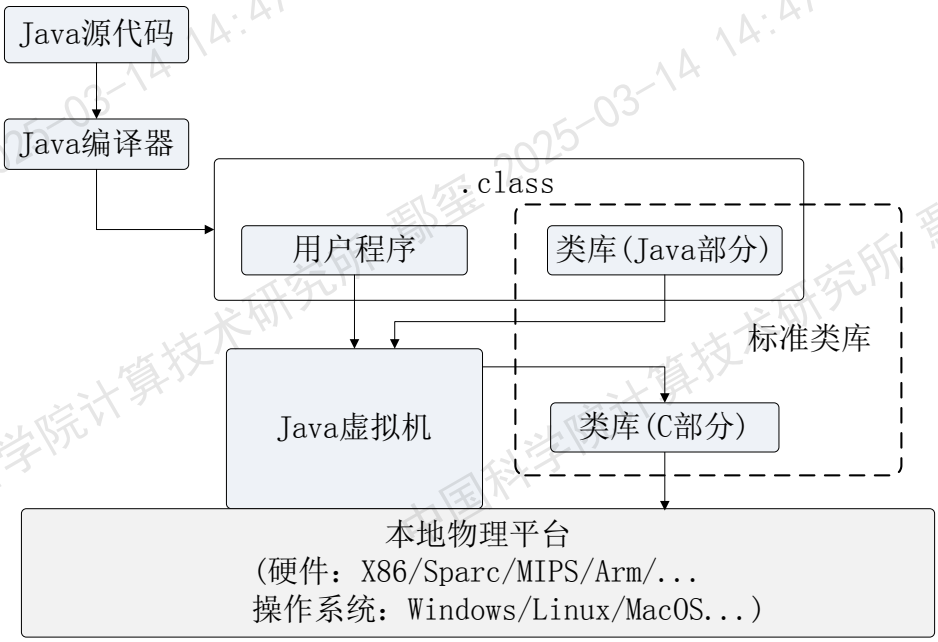

图 1-2 Java 虚拟机外部视图

用户编写的 Java 源程序通过 javac 编译工具编译成为字节码文件(即.class 文件)，与虚拟机提供的类库文件一起，被虚拟机执行引擎直接加载到内存中执行。运行时类库除了 Java 编写的部分，还有部分使用 C 语言编写，根据具体平台编译为相应的动态库。整个虚拟机位于本地物理平台之上，主要由执行引擎和运行时标准类库两部分组成，为用户程序提供运行时支持。

1.2.2　Java 虚拟机的内部视图

Sun 公司提供的官方 Java 虚拟机视图[4]如下所示：

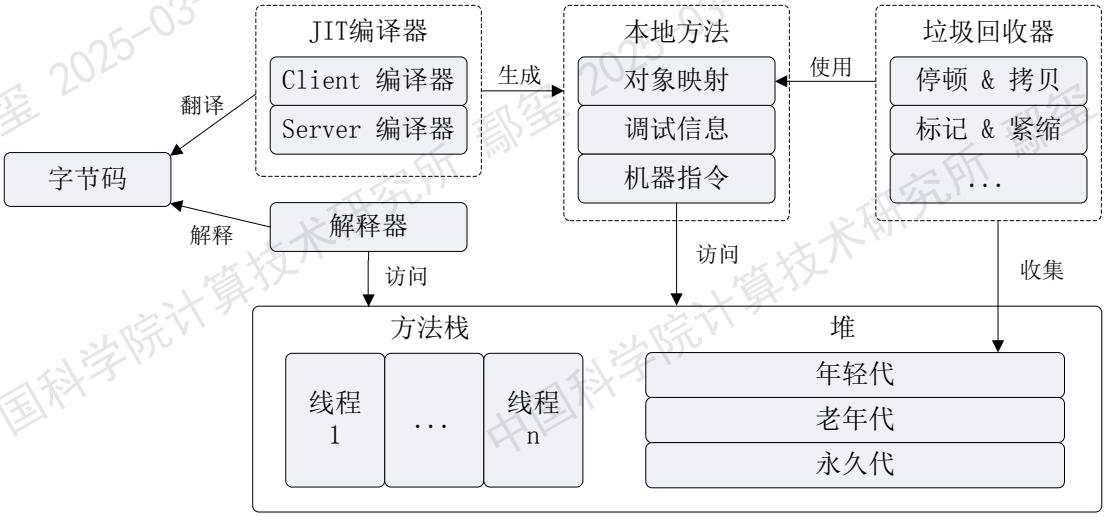

图 1- 3 Java 虚拟机内部视图

Java 虚拟机内部由解释器、即时编译器、本地方法区、垃圾回收器以及堆栈等组件构成。虚拟机执行操作的基本对象是字节码，有解释执行和翻译执行两种模式，分别由解释器和即时编译器负责。现有的虚拟机提供两套即时编译器，即 Client 版编译器和 Server 版编译器，分别适用于客户端和服务器端。即时编译器以方法为单位将字节码进行编译，生成具体平台相关的机器指令，同调试信息与对象映射关系一起存放在本地方法区中。

Java 语言相比 C/C++的一个重要优势在于程序员不需要负责对象内存的分配与释放工作，对象内存的管理工作交给 Java 虚拟机的垃圾回收器完成。现有虚拟机提供了多种垃圾回收器，典型的回收方式包括停顿-拷贝型与标记-紧缩型。这些垃圾回收器在虚拟机执行过程中会使用本地方法区中记录对象的映射信息，识别出活跃对象，对不再使用的内存及时进行回收。

Java 方法运行时使用的主要数据存放在方法的栈和堆中。每个线程有各自独立的方法栈，堆则为各个线程共享。Java 内存模型[5]中的堆是分代的，分配与释放较频繁，生存周期较短的对象被存放在年轻代。经过一段时间仍然存活；生存

周期较长的对象则被存放在老年代。永久代主要存放一些类级对象和类信息本身。无论解释器解释执行过程中还是翻译后的本地方法执行过程中都需要访问方法栈和堆中的数据。

在解释器、即时编译器、垃圾回收器以及堆栈的紧密耦合下，Java 虚拟机为上层的应用屏蔽了底层物理平台特性，抽象出一台平台无关的虚拟的机器，使得 Java 应用能够自如运行于其上。

1.3 MIPS64 体系结构介绍

现代计算机按指令集可分为精简指令集计算机(Reduced Instruction Set Computer, RISC)与复杂指令集计算机(Complex Instruction Set Computer, CISC)。CISC 指令较长，指令功能强大，程序开发效率高，但由于不同的指令需要不同的时钟周期来完成，流水线执行效率较低，以 Intel x86 为典型代表。RISC 仅包含简单、基本的指令，复杂功能通过简单指令的组合完成，由于指令格式规范和执行周期统一，具有高效的流水线操作，典型的 RISC 包括 ARM, MIPS, SPARC 等。

MIPS CPU 是一种 RISC 体系结构 CPU，最早在 80 年代初由斯坦福大学 Hennessy 教授领导的研究小组研制，后来发展为 RISC 工业产品级微处理器，为众多公司用于组建工作站和计算机系统[6]。MIPS 体系结构在设计理念上注重简化硬件，并强调通过软硬件的协同工作来提升性能。MIPS 通用指令集系列由最早的 MIPS I 发展到 MIPS V，嵌入式指令集系列由最早的 MIPS16 发展到 MIPS64，至今已较为成熟。凭借其独特优势，MIPS 逐渐成为市场上最具前景的 RISC 体系结构之一，其产品大多应用在工作站领域。另外，中国国内自主研发的龙芯系列处理器芯片也基于 MIPS 指令集。

MIPS64 提供了 64 位 MIPS 处理器的指令集规范，64 位处理器相比 32 位处理器提高了数据位宽，能够支持更大范围的数据运算和寻址，以及更强的负载能力与扩展性。随着多核处理器的发展，64 位处理器的普及已成为现代计算机架构的主流趋势。

64 位处理器性能需要运行 64 位操作系统及 64 位应用才能充分利用硬件资源，包括 64 位 Java 虚拟机。目前，主流的 JVM 提供了对 x86，ARM，SPARC 等体系结构的支持。近年来，开源 Java 平台 OpenJDK 提供的 Hotspot Java 虚拟机已经成功移植于龙芯硬件平台上[7]，提供对 MIPS64 架构的支持[8]。随着 MIPS64 多核处理器已经应用于服务器和高性能计算机应用中，MIPS64 平台下 Java 虚拟机性能优化研究具有重要意义。

1.4 本文主要工作

如 1.1 所述，Java 虚拟机在现代服务器软件技术层次结构中是承上启下的关键环节。自 Java 语言诞生以来，JVM 性能优化一直是学术研究领域的热点话题。近年来随着用户应用程序规模的增大，应用需求和复杂度的提高，对 Java 虚拟机性能的需求更加迫切。本文的研究目标，是结合 64 位 MIPS 体系结构平台的特点，分析并寻找虚拟机性能的瓶颈进行优化，提升虚拟机性能。

基于上述研究目标，本文研究工作的主要内容如下：

1. 64 位虚拟机方法调用机制的研究

32 位虚拟机向 64 位虚拟机的过渡，伴随着寻址空间的增大以及指针的膨胀。MIPS 体系结构下虚拟机即时编译器中存在大量对 64 位立即数的加载行为，降低了虚拟机性能。本文分析 MIPS64 平台下虚拟机加载的立即数来源，针对方法入口地址一类的立即数提出了异于现有虚拟机系统实现的加载方式，节省了指令条数，提升了性能。

2. Java 虚拟机运行时类库频度的度量研究

现有对虚拟机的优化方法研究大多集中于虚拟机执行引擎和垃圾回收器模块，针对虚拟机运行时类库的优化研究相对较少。本文通过实际用例，对 MIPS64 平台下虚拟机运行时类库的使用频度进行了大量度量和统计，发现了一些典型场景下，存在某些使用频度较高的热点方法，具有潜在的优化空间。

3. 针对类库高频度调用的热点方法的优化方法研究

本文根据运行时类库使用频度的度量统计，提出了使用 JNI 调用机制和使用 Intrinics 机制优化热点方法的方案，并选择典型用例，分别对两种优化方案进行了实现。通过将高频度热点方法进行本地化以及内置于虚拟机中，提高了热点方法的执行效率，从而提升系统性能。

本文的研究工作均在龙芯 3 系列软硬件平台上进行了测试和验证，提出的立即数加载方案以及热点类库优化方案均带来大约 5%性能提升。本文相关优化成果已经集成应用于龙芯最新 JDK 产品中。

1.5 本文组织结构

本文共分六章。

第一章介绍了现代服务器软件的主流技术架构，分析了 Java 虚拟机在服务器领域的重要作用以及 MIPS64 平台下应用对 Java 虚拟机性能的迫切需求。给出本文研究的意义和目标。

第二章介绍了 Java 虚拟机的主要结构和工作原理。重点对虚拟机解释器、即时编译器和垃圾回收机制进行了深入剖析，并对当前 Java 虚拟机领域的重要优化方法进行了总结。

第三章介绍了针对虚拟机即时编译器部分进行的方法调用机制优化。首先分析了 MIPS64 系统下虚拟机因指针膨胀而产生的性能瓶颈，提出一种通过在内存中增设立即数区缓存 Java 方法入口地址的方法，将多条立即数加载指令转化为一条访存指令，从而提升性能。随后针对优化方案进行了系统设计与算法实现。

第四章介绍了针对虚拟机运行时类库部分进行的高频度热点方法优化。首先对 Java 虚拟机运行时类库方法的使用频度进行了度量。根据统计结果，针对高频度调用的“热点”方法，通过 Java 虚拟机提供的 Intrisics 机制以及 JNI 调用机制，将“热点”方法本地化，从而改善方法执行效率，提升性能。

第五章对龙芯软硬件平台进行了介绍，并分别针对 64 位立即数加载优化方案和运行时类库热点方法优化方案在龙芯平台上进行了效果验证，并通过 SpecJVM 2008 测试集对性能优化幅度进行了测试。

第六章对全文研究工作成果进行了总结，并针对研究中遇到的一些问题展开进一步探讨，为后续相关研究提供了参考方向。

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

第二章　　Java 虚拟机性能优化研究

Java 虚拟机作为支撑 Java 应用的基础平台，为实现 Java 程序跨平台运行，对上层应用隐藏了底层技术的复杂性以及机器与操作系统的差异性，提供了一整套复杂的执行引擎和运行时支持框架[9]。同时为了提高 Java 程序的执行效率，多年来国内外研究者投入了大量精力对 Java 虚拟机的核心组件进行性能优化，积累了大量成果与经验。了解 Java 虚拟机的结构、工作原理与重要优化方法，对进一步改善 Java 虚拟机的执行效率，挖掘性能提升空间有重要指导意义。

2.1 Java 虚拟机执行框架

Java 虚拟机的运行主要依赖解释器、即时编译器、垃圾回收器和堆栈的协同工作[10]。其中，解释器与即时编译器构成了 Java 虚拟机执行引擎的核心。

Java 虚拟机运行的基本框架如下图所示：

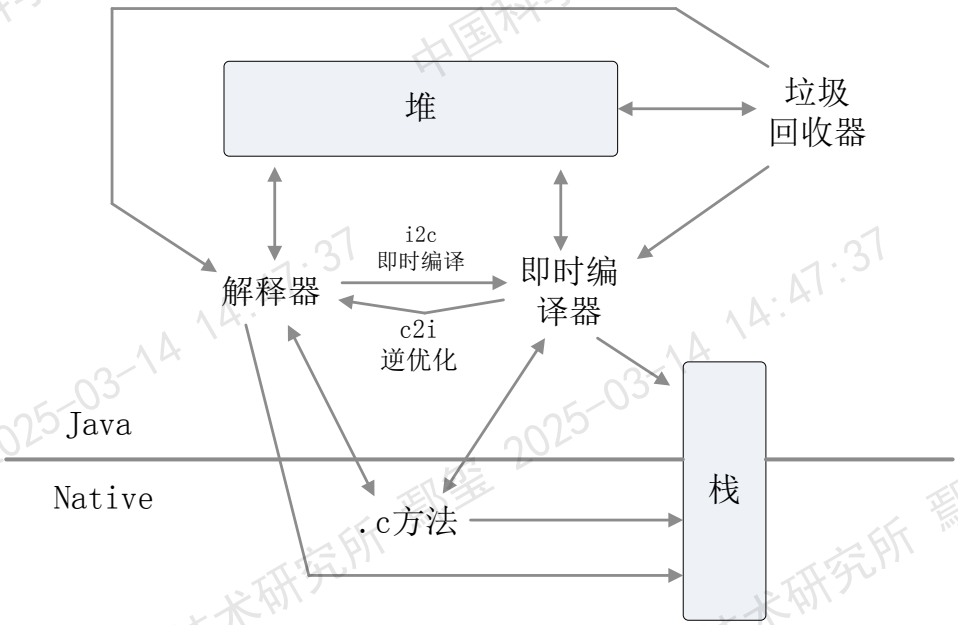

图 2-1 Java 虚拟机全景图

如 1.1.2 所述，Java 程序的执行主要有解释执行与翻译执行两种模式。Java 程序开始运行时先由解释器取字节码逐条解释执行，对于调用频度达到一定阈值的方法，即时编译器以方法为单位将整个方法的字节码序列翻译成为平台相关的机器指令序列，保存在本地方法区中，后续对该方法的调用会直接执行翻译后的本地代码。由于翻译后代码无论从执行效率还是优化程度都远高于解释执行，因

此能达到更高的性能，但前期需要有对整个方法的编译开销。另外，现代虚拟机即时编译器对翻译后代码进行了部分激进优化，当适用条件发生变化时，翻译后的代码会失效，这种情况下虚拟机能够退回解释模式由解释器继续负责执行，这一过程称为“逆优化”。

无论解释器还是即时编译器，执行时都会在堆栈中存放数据。栈用于保存方法的局部数据，而运行时动态创建的对象则在堆中分配。垃圾回收器负责在整个 Java 程序执行过程中回收和整理堆中对象占用的内存。

除了 Java 方法，JVM 还提供了支持 Java 方法调用 C/C++方法的机制。Java 虚拟机提供了一套 Java 本地接口规范来实现这一功能，使得解释器和即时编译器执行过程中能够调用 C/C++编译器编译后的本地动态库中的方法。这一过程中涉及 Java 运行环境和本地运行环境的转换，主要是栈帧格式的转换。这一过程也是由 Java 虚拟机负责完成的。

2.2 解释器及其性能优化

2.2.1 解释器工作原理

解释器负责 Java 方法的解释执行，执行的基本单位是一条字节码。字节码是 Java 虚拟机的基本指令格式，除了少数需要参数的字节码，大多数字节码都是一个字节长。Java 虚拟机规范中给出了 200 多条字节码，主要分为如下几种类型:

表 2-1 字节码分类及示例

字节码类型	代表字节码示例
存储指令	aload_0, istore
算数与逻辑指令	ladd, fcmpl
类型转换指令	i2b, d2i
对象操作与创建指令	new, putfield
堆栈操作指令	swap, dup2
控制转移指令	ifeq, goto
方法调用与返回指令	invokespecial, areturn

Java 虚拟机字节码是基于栈结构执行的。指令的操作数存放在栈中，指令通常最多访问栈顶的两个操作数。解释器通过一个字节码指针(Byte Code Pointer,

BCP)指向将要执行的字节码指令。解释器根据 BCP 指向的具体的字节码按照 Java 虚拟机的字节码规范，实现相应的语义动作，操作的源数据和结果均存放在栈或堆中。每执行完一条字节码，BCP 会自动后移，指向下一条待执行的指令继续执行，跳转指令可以强行修改 BCP 的值。这一过程中 BCP 的功能类似 CPU 的 PC 寄存器。

2.2.2 解释器性能优化

早期的解释器实现是平台无关的，其核心是一个 switch-case 分支结构。针对每一中字节码实现一个对应语义的处理函数。每次运行时根据字节码的值跳转到不同的分支，执行相应的处理函数。其优点是结构简洁，清晰明了，且平台无关，移植性好。然而每条字节码的执行需要译码和跳转调度的开销较大，影响了执行效率，且优化空间仅限于每条字节码处理函数的内部，性能较低。

对性能的进一步挖掘促进了基于模板的解释器的诞生。其优化思想是将平台无关解释器中的频繁执行环节，使用汇编语言实现。模板解释器中，每条字节码对应一段用汇编语言实现该字节码语义的最小规模指令序列，这段指令序列称为模板。在使用汇编语言的基础上，可以进一步深入利用底层硬件的资源，比高级语言更容易挖掘性能，因而获得了更高的执行效率。

线索化模板解释器与非线索化解释器的对比如下图所示：

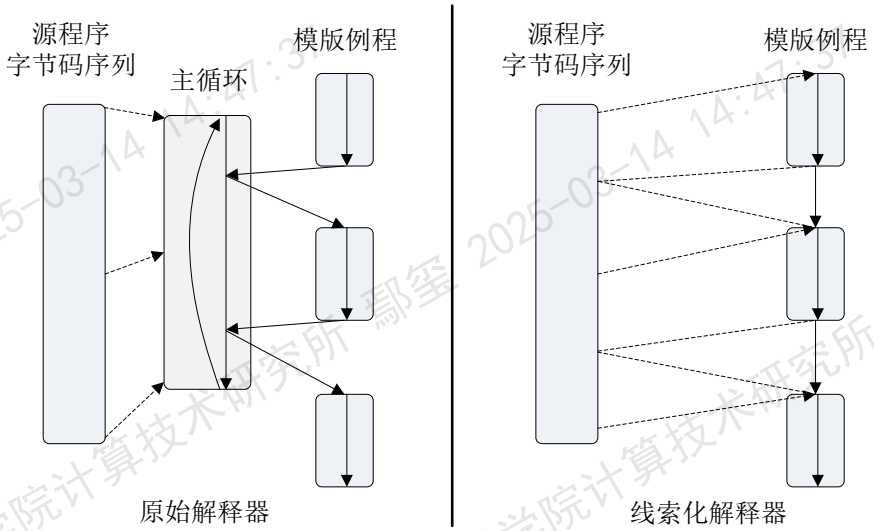

图 2-2 线索化解释器与原始解释器对比

由于 switch-case 结构导致的大量间接跳转影响了译码与分发的性能，可以将集中式分发转化为在每个模板中进行分布式分发，即每个模版在执行结束后执行自带的一段分发指令序列，从而确定下一条要执行的模板位置。这样通过在每个

模板冗余一段分发指令序列的方式，去除了集中式的 switch-case 结构，并减少的跳转开销。这样的解释器称为线索化模板解释器。

随着线索化模板解释器的平台相关性增强，可以在寄存器层面上进一步优化模板。尤其对于 RISC 体系结构，如 MIPS。这类体系结构具有大量寄存器的优势，因而可以选定若干寄存器专门用于保存频繁访问的信息，进一步减少指令条数。例如，使用专门寄存器保存 BCP 的值，能够加速对 BCP 信息的访问；可以将虚拟机栈顶的数据用专门的寄存器保存，从而加速堆栈的模拟等。

尽管现代 JVM 的解释器相比最原始的版本已经有了大幅度的性能提升，但由于其以字节码为单位，需要进行重复的解析与解码操作，且对于操作数栈的访问只能从一端进行，限制了数据吞吐且不利于优化。因此相比 C/C++等静态编译型语言仍有大量性能差距。

2.3 即时编译器及其性能优化

早期的 JVM 执行引擎采用纯解释器实现，执行效率低，性能与 C 语言相差几十倍。为了弥补与静态编译型语言的巨大性能差距，动态编译技术应运而生。即时编译器的提出极大程度上促进了 Java 的发展与应用。

2.3.1 即时编译器的工作原理

在 Java 虚拟机中，即时编译器[11](Just-in-time compiler, JIT compiler)是一个把 Java 字节码序列转换成处理器可以直接执行的机器指令的模块。

虚拟机在执行过程中具有剖析功能，能统计当前执行方法的调用次数，当发现某个方法或代码块的运行非常频繁时，就会把这些代码识别 为“热点代码”。即时编译器能够在 JVM 运行期间，通过专门的编译线程把热点代码编译成本地具体平台的机器指令序列，并做各种层次的优化。

即时编译器编译后生成的本地代码会被虚拟机缓存起来，由于本地执行效率远远高于原字节码序列的解释执行，并且编译期间经过了充分的代码优化，因而能够获得显著的性能提升[12]。

即时编译的本质是利用程序的局部性原理，Java 应用运行的绝大多数时间是在执行少数方法。因此可以针对高频执行的方法进行动态编译，在编译代价和性能回报之间找到恰当的平衡，Java 虚拟机即时编译器执行框架如图 2-3 所示。

Java 应用程序启动时默认先由解释器取程序的字节码逐条执行。虚拟机内部会为每个方法建立并维护计数器，统计方法的执行次数。方法每被调用一次，对

应的计数器的值就会加 1。随后判断当前方法的被调用次数是否达到了某个阈值。以 OpenJDK Hotspot 虚拟机为例，其 Client 编译器默认阈值为 1500 次，而 Server 编译器默认阈值则为 10000 次。

如果当前待执行方法被调次数没有达到阈值，则继续由解释器负责解释执行；如果达到了阈值，那么该方法将被识别为热点方法。首次被识别的热点方法尚未被翻译，将被加入到虚拟机维护的待编译的方法队列，等待与当前 Java 线程并行的翻译线程处理。

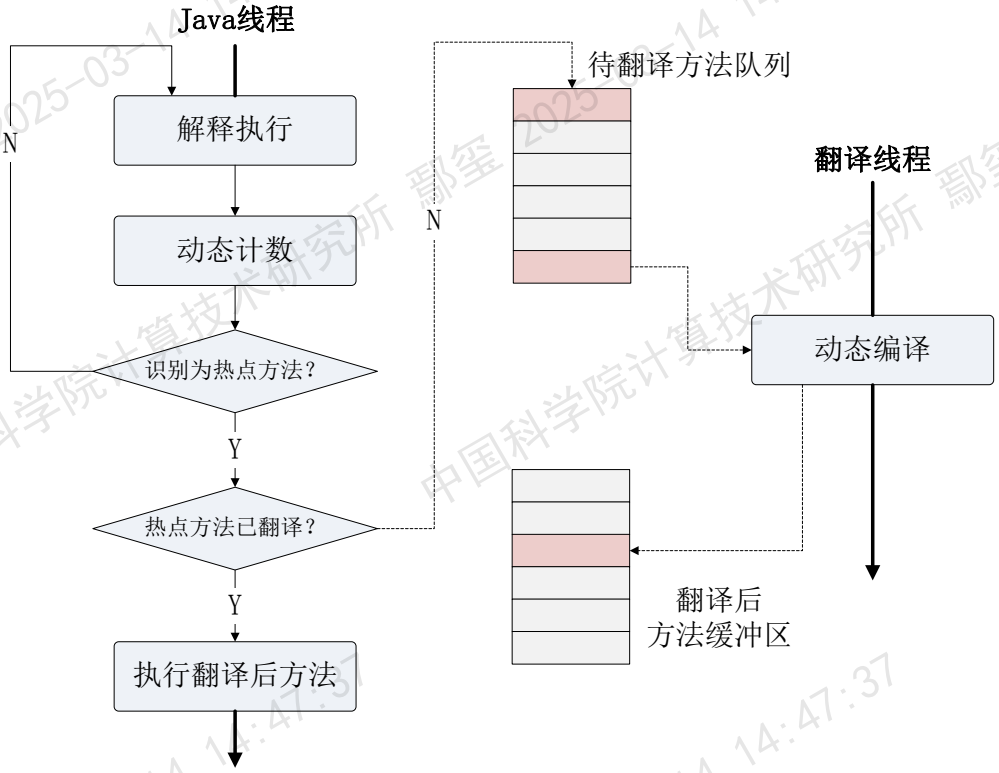

图 2-3 Java 虚拟机 JIT 编译器执行框架

翻译线程从待编译方法队列中取出待编译方法，按照编译理论的基本流程逐步将整个方法的字节码序列转化成目标机器代码[13]。这一过程经历了词法分析、语法分析、语义分析、代码优化、中间代码生成、目标代码生成等一系列繁琐的过程，最终得到的翻译后的本地代码被存放在虚拟机内部维护的被称为方法缓冲区(Code Cache)的内存区域中。

后续执行过程中再次调用该方法，虚拟机将判断出该方法已经具有翻译后版本，从而直接从翻译后代码缓冲区中取出方法的机器指令序列直接执行。对于新的被调方法，如果尚未被识别为热点方法，则会重新退回解释器执行。整个虚拟机执行框架中，解释器与编译器正是按照上述方式相辅相成地协同工作的。

2.3.2 即时编译器的代码优化

在即时编译器后台执行编译的过程中，Client 版本与 Server 版两个版本的编译流程是不同的。以 Client 版本编译器为例，它完成的是一个三段式编译过程[14]，主要进行局部性优化。

Client 版编译器翻译过程如下图所示：

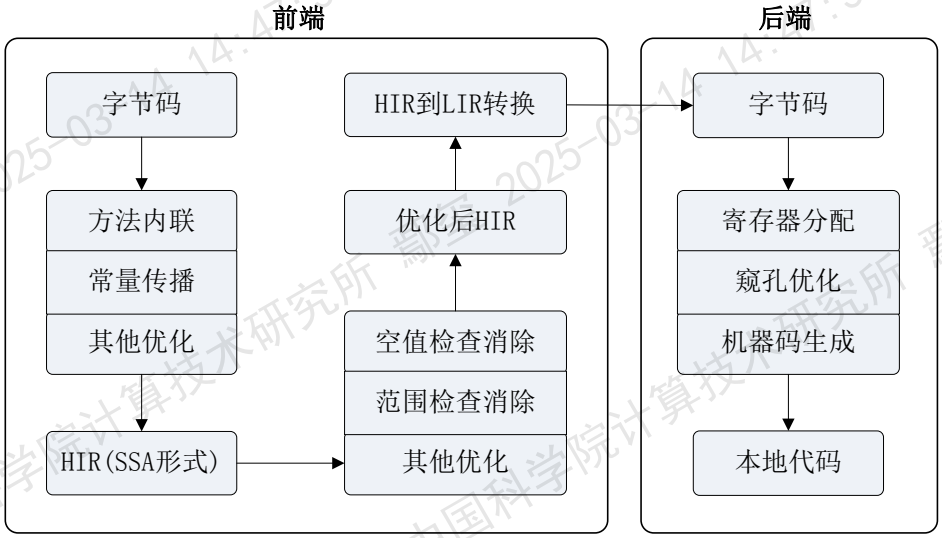

图 2-4 Client 版即时编译器基本架构

第一阶段，编译器前端将字节码转换成一种高级中间代码表示形式(High-Level Intermediate Representation，HIR)。HIR 使用静态单分配(Static Single Assignment,SSA)的形式来代表码值，使之后进行的优化动作更容易实现。在 HIR 生成前会对一些方法进行内联并开展常量传播等优化工作。

第二阶段，编译器后端从 HIR 中产生低级中间代码表示(Low-Level Intermediate Representation，LIR)，在转换之前会在 HIR 形式上进行空值检查消除，范围检查消除等另一部分优化。

第三阶段在后端进行，使用线性扫描算法(Linear Scan Register Allocation)在 LIR 上进行寄存器分配，并在 LIR 上进行窥孔优化，最后生成机器代码。

Server 版编译器[15]则是专门面向服务器端的典型应用，为服务器端的性能配置进行过专门调整，其优化深度接近达到 GNU C++编译器使用-O2 参数的级别，几乎会执行所有的经典优化动作。包括：循环展开、无用代码消除、循环表达式外提、公共自表达式消除、常量传播、基本块重排序等。其寄存器分配采用全局图着色算法，充分利用了 MIPS 等 RISC 架构的大量寄存器集合。

2.3.3 即时编译器的运行时优化

即时编译器不仅能够应用静态编译器中的经典优化方法对翻译后代码进行优化，还能够利用在运行阶段动态编译的特点，收集运行时信息，根据实际运行时的特性进行分析并进行更深层次的优化。典型的分析技术包括类型继承关系分析与逃逸分析。

类型继承关系分析技术用于判定某个类是否存在子类，子类是否为抽象类等信息，并由此进一步分析当前已加载的类或接口是否包含两个或两个以上的实现。根据类型继承关系分析的结果可以对一些类做激进的内联优化。

传统编译器对虚方法是不能直接进行内联的，因为虚方法的多态特性使得静态编译时刻无法确定实际执行的对象所使用的方法。但即时编译器中，能够根据继承关系分析结果确定一个虚方法是否有多个目标版本进行选择。如果查询结果只有一个版本，则编译时同样可以进行内联，只要程序后续执行过程中没有加载到会使该方法接收者继承关系发生变化的类，那么该版本就可以一直使用。

如果该方法后续执行过程中触发了多态行为，方法接收者继承关系发生变化，虚拟机能够禁用先前进行内联的版本，退回解释模式继续执行，从而为这种激进的优化预留的一个“逃生门”，保证方法能够正确执行。

逃逸分析技术能够分析对象的作用域。当一个对象在方法中被定义后，可能被外部方法所引用的情况称为逃逸。例如，方法中定义的对象可能作为参数传递到其他方法中，这种情况称为方法逃逸；还有一些对象甚至可能被外部线程访问到，这种情况称为线程逃逸。

虚拟机运行过程中能够识别出不会逃逸到方法或线程之外的对象。这些对象别的方法或线程无法通过任何途经访问到这个对象，从而能够对这些对象进行高效优化。典型的优化方法包括栈上分配、同步消除和标量替换。

栈上分配是针对不会出现方法逃逸的对象，将在堆上分配对象改为在栈上分配的优化方法。改为栈上分配后，方法结束时对象占用的空间会随栈帧出栈而自动销毁，大大减少了垃圾收集系统管理堆内存的压力。

同步消除是针对不会逃逸出线程的对象，可以确定对这些对象的读写不会存在竞争，从而消除对相关变量实施的同步措施。

标量替换是针对不会方法逃逸并且能够拆分的对象。程序运行时将不再创建该对象，改为直接创建该对象的若干被当前方法引用的成员变量。拆分后将能够将对象的成员变量在栈上分配，也能够为后续的进一步优化手段创造条件。

即时编译技术能够以运行期性能监控为基础进行优化，使得 Java 语言具相比静态编译型语言具有独特的优势，性能差距显著缩小，极大程度上提高了 Java

的实用性。

在 MIPS 等 RISC 平台下，能够利用平台特性对虚拟机即时编译器和运行时模块进一步优化。利用通用寄存器多的独特优势，可使用专门的寄存器保存频繁访问的运行时信息，从而减少运行时开销，进一步加速执行效率。另外，可以借助具体硬件平台提供的特殊指令来完成虚拟机的特定功能，例如读取数据的原子性等，相比平台无关的软件加锁方式，进一步提高了虚拟机性能。

2.4 垃圾回收机制

Java 并不像 C++等语言那样，提供“delete”关键字，这使得应用程序中无法显式地释放对象。运行环境中无用内存的识别与回收工作是由 Java 虚拟机在后台自动完成的，这一过程称为垃圾回收(Garbage Collection, GC)。

现代 JVM 的垃圾回收器一般使用根搜索算法来判别堆中对象是否存活。基本思路是先确定一个 GC 根对象集合，这一集合包括虚拟机栈中引用的对象、类静态属性引用的对象，方法区常量引用的对象以及本地方法栈中引用的对象。以根对象集合中的对象作为起始点递归向下搜索，搜索走过的路径称为引用链。当对象到 GC 根对象集合没有任何引用链相连时，则此类对象是不再会被引用到的，即判定为无用对象。

对于无用对象的回收有多种算法。常用算法包括标记-清除算法、复制算法与标记-整理算法[16]。几种算法的工作方式对比如下所示：

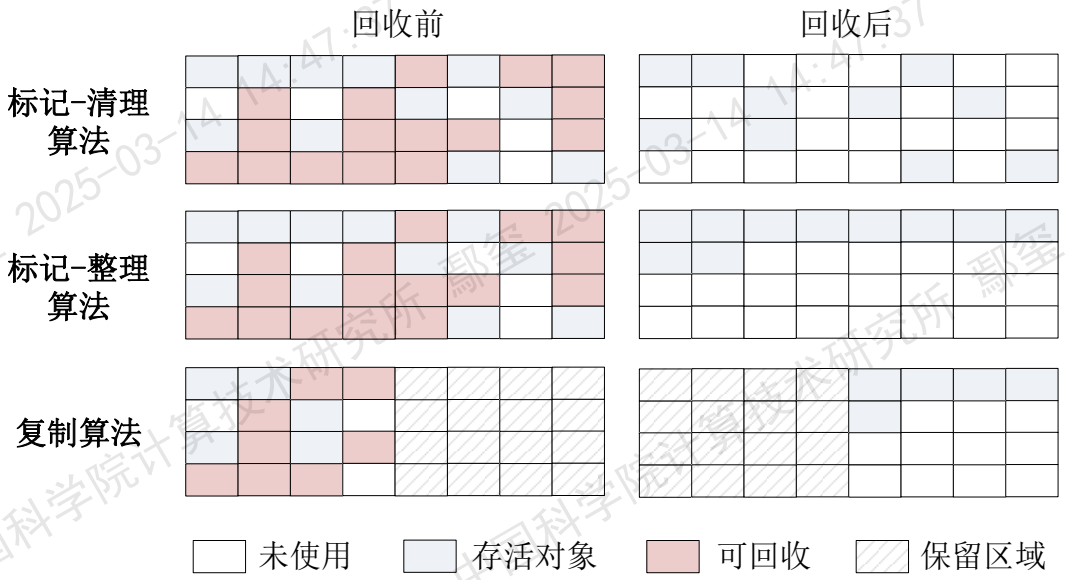

图 2- 5 三种垃圾收集算法对比

标记-清理算法首先标记出所有需要回收的对象，标记完后统一回收。这种

方法简单直观，但缺点是清除无用内存后产生大量不连续的内存碎片，效率不高。

标记-整理算法的标记过程同标记-清理算法相同，但随后让所有存活的对象都向一端移动，然后直接清理掉端边界以外的内存。其优点在于保证了可用内存的连续性，避免了碎片的产生，缺点在于大量对象的移动操作需要较大的时间开销。

复制算法将可用内存划分为相等的两块，一次只使用其中一块。当本块内存用完，则直接将该块内存中的存活对象复制到另外一块上，然后把已使用过的内存空间一次性清理掉。这样内存分配不用考虑内存碎片的复杂情况，清理对象的效率也较高，缺点在于代价是将可用内存缩小为了原来的一半。

为了进一步提高垃圾回收的性能，现代虚拟机对堆内存进一步分为年轻代和老年代。年轻代保存生存周期较短频繁分配释放的对象。老年代保存生存周期较长的对象。对年轻代堆内存采用较高的 GC 频率，对老年代对内存采用较低的 GC 频率。分代回收的优点在于提高了堆中不同区域的利用率，并且减少了垃圾回收时遍历堆内存的范围，提高了回收效率。

垃圾回收器按照 GC 过程本身是否多线程分为串行 GC 与并行 GC。串行 GC 实现简单但无法利用现代服务器的多 CPU 资源，单次 GC 时间较长。随着多 CPU、多核技术的普及，大量串行算法被改造为并行算法，使得 GC 工作由多个 GC 线程并行完成，GC 时间得到线性缩短。

串行 GC 与并行 GC 执行过程中，均需要所有执行中的 Java 线程暂停。为了加快服务器应用程序的实时响应，出现了并发 GC。并发 GC 将整个 GC 过程做了进一步细分，使得 GC 期间的标记等阶段 Java 线程能够与 GC 线程并行执行。由于 Java 线程在执行过程中可能会对堆中对象的活跃情况产生影响，因此在回收之前仍需要暂定 Java 线程并进行重新标记，这种方式进一步提高了 Java 虚拟机的实时性能。

内存管理与垃圾回收在常常是影响虚拟机性能和并发能力的关键因素。虚拟机发展过程中，针对 GC 过程也投入了大量优化与组合调优。没有通用意义上的最佳方案，需要根据实际需求配置最适合的回收方式，才能使性能达到最优。

2.5 本章小结

本章剖析了 Java 虚拟机的基本工作机制，对解释器、即时编译器以及垃圾回收器的工作原理进行了深入介绍，分析了解释器和即时编译器发展过程中的重要优化方法。重点对即时编译器的代码优化技术和运行时优化技术做了介绍和总结。解释器与即时编译器在 Java 虚拟机工作过程中各有优势，协同工作，保留

了 Java 语言的跨平台特性的同时，弥补了相比 C/C++静态编译的性能差距。

解释器、静态编译器、即时编译器的特性比较总结如下表所示：

表 2- 2 解释器、静态编译器、即时编译器比较

比较角度	解释器	静态编译器	即时编译器
复杂度	低	高	中
性能	低	高	中
指令规模	最小	最大	中
实时性	高	低	中
平台移植性	高	低	低
典型语言	Basic, Lisp, Perl, Python Ruby, JavaScript, SQL, Matlab, Unix Shell	C/C++, Pascal, Fortran	Smalltalk, Self, Java
优化理论	相对较少	相对较多	相对较多
适用场合	交互型事务 新语言的快速原型验证 对平台性要求高、对性能要求不敏感的场景	计算密集型事务 对峰值性能要求高的场景	兼顾交互实时性与峰值性能 只编译程序中运行频度最高的成分，以较少翻译代价换取性能提升

Java 虚拟机作为发展多年的大规模系统软件，对其性能优化的研究仍在未停止，尤其是结合特定体系结构对即时编译器进行平台相关优化仍然有提升翻译后代码质量的空间。此外，当前的优化方法多集中在虚拟机执行引擎部分，对于 Java 虚拟机类库部分的性能度量与优化投入的研究较少。针对这即时编译器与运行时类库的性能度量与瓶颈分析有较大的研究意义，能够指导开发者进一步挖掘 Java 虚拟机的性能优化空间。

第三章　64 位虚拟机方法调用优化

执行引擎是 Java 虚拟机的核心。大多数 Java 程序在虚拟机上一般默认以混合模式运行，即初期以解释模式运行，当方法执行达到一定频度由即时编译器编译执行。对于现代应用，尤其是服务器应用，即时编译器编译执行占总执行时间相当大的比重。因此即时编译器编译生成的代码质量对虚拟机性能影响显著。随着 32 位虚拟机向 64 位虚拟机的过渡，指针膨胀引发了即时编译器翻译后代码执行效率的降低，在 MIPS 等 RISC 平台下出现了新的性能瓶颈。

3.1 研究背景

3.1.1 32 位虚拟机与 64 位虚拟机对比

随着主流 CPU 支持 64 位架构，Java 虚拟机也在很早之前推出了支持 64 位系统的版本，然而 Java 程序在 64 位 JVM 上运行需要付出较大的额外代价。

首先是内存问题，由于指针膨胀和各种数据类型对齐补白的原因，运行于 64 位系统上的 Java 应用需要消耗更多的内存，通常要比 32 位系统额外增加 10%~30%的内存消耗[17]。

其次是 JIT 编译器生成的机器代码的执行效率问题。64 位指令集中，由于存储地址范围扩大，导致翻译后方法中存在立即数膨胀、指针膨胀问题,即立即数和指针占用的字节数更多，加载更加复杂，导致翻译后的指令条数远远大于 32 位指令集[18]。

对 32 位和 64 位虚拟机的性能评测结果显示[18]，64 位 JVM 的运行速度在各个测试用例上基本全面低于 32 位 JVM，存在大约 15%的性能差距。随着硬件进一步发展，计算机将全面过渡到 64 位架构，主流的应用也会从 32 位向 64 位发展，虚拟机也会进一步完善对 64 位的支持。

3.1.2 MIPS 体系结构 64 位立即数加载方式

在 MIPS 等 RISC 体系结构中，由于指令定长的限制，将一个 64 位立即数加载到寄存器中需要多条指令。以 MIPS 为例，由于 MIPS 指令长度固定为 32 位，因此对于立即数加载指令，除去操作码、源寄存器以及目标寄存器占用的位数，留给表示立即数的位数仅有 16 位，即一条立即数加载指令一次只能将 16 位立即

数载入寄存器。64 位立即数要加载到寄存器中需要借助移位操作完成[19]。

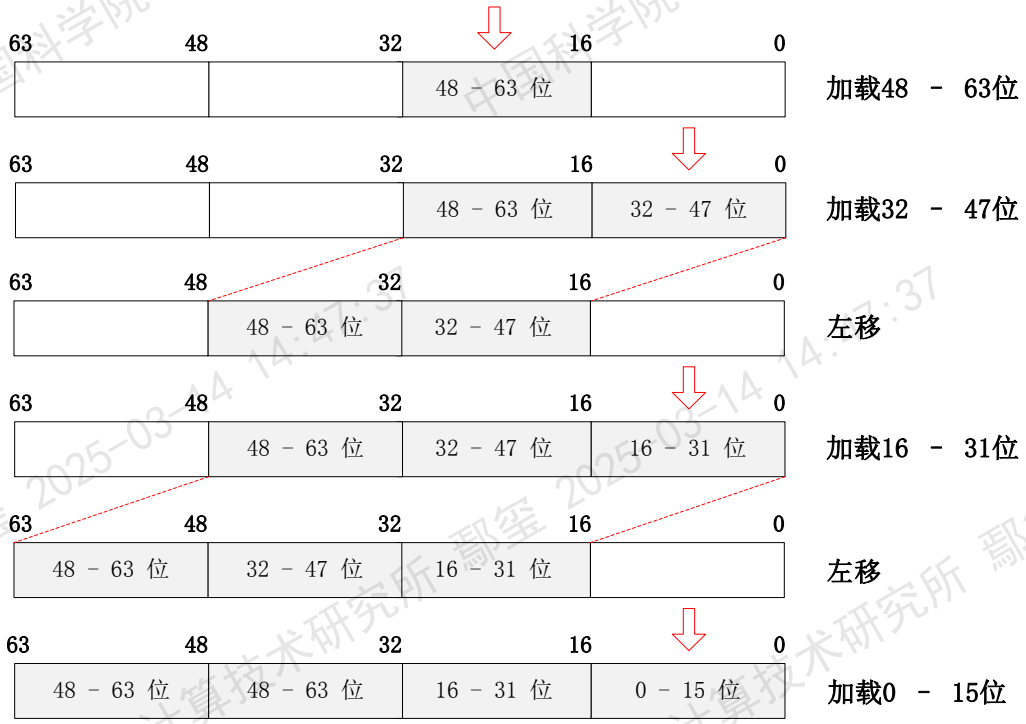

图 3-1 MIPS 体系结构 64 位立即数加载过程

MIPS64 平台下立即数加载过程如上图，首先将原立即数的 48~63 位加载到寄存器的 16~31 位。随后将原立即数的 32~47 位加载到寄存器的 0~15 位。将寄存器中的值左移 16 位后，原立即数的 32~63 位存在寄存器的 16~47 位，低 16 位被腾出来。这时再将原立即数的 16~31 位加载到寄存器的低 16 位，然后将寄存器左移 16 位，这时原立即数的 16~63 位已经被加载到寄存器的 16~63 位。最后再将原立即数的 0~15 位加载到寄存器的 0~15 位，这样就完成了加载。

在 MIPS 中，上述过程通过如下 6 条指令序列实现[20]：

```
lui  rd, imm >> 48
ori  rd, imm >> 32
dsll rd, rd, 16
ori  rd, imm >> 16
dsll rd, rd, 16
ori  rd, imm
```

图 3-2 MIPS64 立即数加载指令序列

其中，rd 为目标寄存器，imm 为待加载的 64 位立即数。ori 指令用于将 16 位立即数加载到寄存器的 0~15 位，lui 指令用于将 16 位立即数加载到寄存器的 16~31 位，dsll 用于实现左移操作。

3.1.3 64 位虚拟机中的立即数来源分析

Java 虚拟机中翻译后代码中存在的立即数主要包含两类。

一类是跳转目标地址。如 2.3 所述，虚拟机翻译过程包括解析生成中间代码,机器无关优化,指令选择,全局代码转移和调度,窥视孔优化和代码生成等多个阶段。由于指令中存在大量的分支跳转指令，因此会产生大量跳转目标地址。这些跳转地址一般为 Java 方法翻译后生成的方法入口地址，或虚拟机运行过程中生成的具有特定功能的公共例程入口地址。

另一类是对象的引用。Java 方法中会创建大量对象，对这些对象的操作是通过对象的引用来进行的。在即时编译器翻译后代码中，这些堆中数据对象的引用，体现为大量常数，从而翻译生成的指令中也包含大量这类立即数的加载操作。

这两类立即数中，方法的入口地址具有更强的确定性，除了激进优化的部分方法，大多数 Java 方法经过翻译后生成的本地代码不会更改，因而方法的入口地址在本地方法生成时已经确定，并且在随后应用程序执行过程中会反复调用。因此，跳转目标地址类型的加载作为被高频执行的操作，对虚拟机性能具有更显著的影响。

3.2 原有方法调用机制

3.2.1 现有虚拟机方法调用及解析过程

Java 虚拟机规范中定义了 invokestatic、invokespecial、invokevirtual、invokeinterface 四种字节码，分别针对不同类型的方法调用。具体如下：

表 3- 1 JVM 规范提供的方法调用相关的字节码

字节码	功能
Invokestatic	用于调用静态方法
Invokespeiall	用于调用父类构造方法、初始化方法或私有方法
Invokevirtual	用于调用对象实例方法
invokeinterface	用于调用接口方法

即时编译器在编译时根据 JVM 对字节码规范的定义，按相应的翻译规则转换为本地指令序列，实现相应的语义动作。由于 Java 虚拟机中的类是运行过程中按需动态加载的，并且存在多态的特性，因此编译时刻不能确定被调方法是否已经被虚拟机加载，实际的被调方法入口地址需要在首次运行时通过解析过程完

成。解析过程如下图所示：

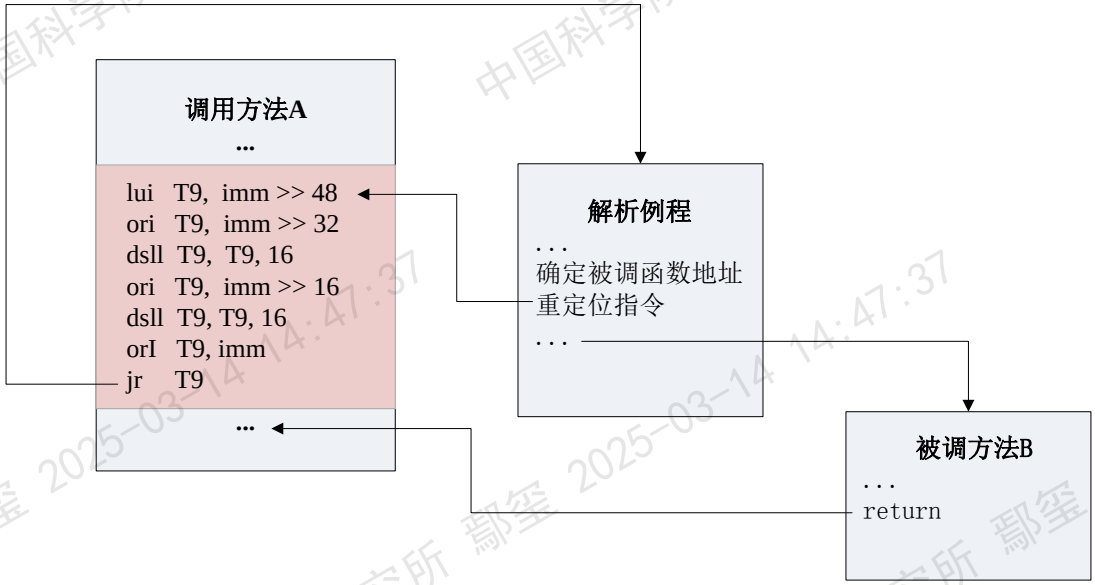

图 3-3 现有 MIPS64 虚拟机方法调用解析过程

现有虚拟机的做法是首先在调用处生成指令序列加载一个特殊的解析例程入口地址 imm 到通用寄存器中(图中为 T9 寄存器)，然后生成转向这个地址的跳转指令。这样第一次执行这段翻译后代码时就会跳转到解析例程执行，解析例程分析当前虚拟机中类的加载情况和类继承关系能够确定被调方法 B 的入口地址，这时会将先前生成的指令序列中的解析例程地址改为实际被调方法的入口地址。这样后续再执行方法 A 时在调用处就能够直接正确跳转到方法 B 了。

3.2.2 现有虚拟机方法调用机制性能分析

现有的方法，加载一个 64 位方法入口地址需 6 条指令。并且这 6 条指令之间存在数据相关，因而执行代价较高。在虚拟机中， 有一类公共例程，用于执行确定的功能，例如方法调用类的解析、执行方法出错时的处理等等。这些例程的执行代码在虚拟机启动时就已经生成，例程的入口地址均为常数。在执行过程中会被反复使用，因而产生大量 64 位立即数的加载。

另一方面，在 64 位虚拟机运行过程中，Java 方法翻译后生成的本地代码方法入口，在每次调用时都会作为跳转目标地址加载到寄存器中，因而观察虚拟机的翻译后代码片段，大量方法入口处会包含这样的立即数加载序列。相比 32 位虚拟机，翻译后产生的指令条数会大大增加，不仅占用了更大的存储空间，也降低了执行效率。

针对方法与公共例程入口地址， 现有虚拟机的处理方式是全部通过立即数

加载指令和左移指令加载。即使同一个地址在两个不同的方法中被调用，该入口地址也会在两个方法的翻译后代码中分别以操作数的形式各出现一次。综合上述分析，本文提出另一种加载虚拟机中方法入口地址到寄存器中的方式，尝试通过在虚拟机进程中开辟专用内存空间保存方法与公共例程入口地址的方式，将立即数加载操作转化为访存读数操作，从而减少指令条数，提升性能。

3.3 系统设计

3.3.1 静态数据结构

本文提出的方案是在虚拟机原有结构的基础上，增设一项全局数据结构，称为立即数表，用于保存公共例程的入口地址以及 Java 方法翻译后代码的入口地址。运行过程需要使用入口地址时，从立即数表中查出对应的表项，直接从立即数表中读取即可。

公共例程的入口地址在虚拟机启动后便会生成并不再改变，并且只有固定有限的几种。这些地址可以存放在立即数表的特殊位置，查询的时候按照存放的约定读取即可。对于 Java 方法的翻译后本地代码入口地址，在方法翻译完成阶段便可确定，除了少数做激进优化的被翻译方法，大多数方法的入口地址也不会改变。

本文提出的全局立即数表的结构设计如下所示：

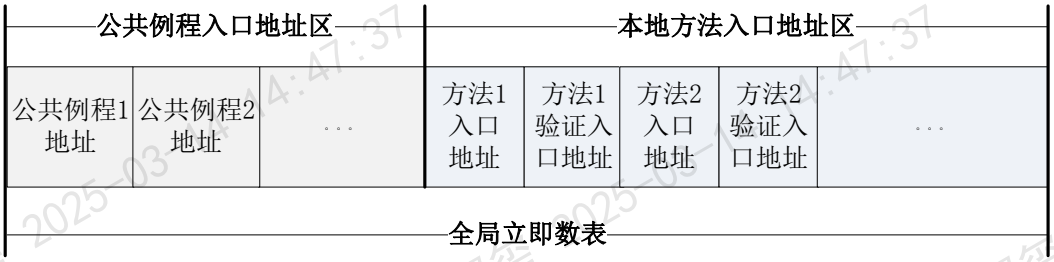

图 3-4 全局立即数表结构设计

全局立即数表分为两个区域，公共例程入口地址区和本地方法入口地址区，分别用于存放虚拟机公共例程入口地址和即时编译器编译后生成的本地方法入口地址。Java 虚拟机中每个被即时编译的方法有两个入口地址，一个普通入口地址和一个校验后入口地址。

由于 Java 中方法存在多态的特性[21]，在运行时根据具体的对象类型才能确定要调用的是哪个类的方法。因此在每个方法的入口处会有一段校验代码，这段代码会比较实际调用方法的对象所属的类和当前翻译后代码所属的类是否一致[22]。如果一致，则跳转到方法的可执行代码入口处，也就是验证后入口地址处

执行，如果不一致，则会跳转到处理当前方法失效的公共例程去解析实际需要调用的方法。

对于静态方法或者类已经被加载到内存中的 final 方法，这类方法在后续的实际运行过程中不会再表现出多态的行为，实际调用的对象的方法与当前翻译后的本地代码一定是一致的。因此对于这类情况无需再执行方法入口处的校验代码。方法在第一次被调用时会被例程解析[23]，解析例程会根据当前方法和类的加载情况分析当前方法是否会表现出多态行为。如果判定当前方法不会出现多态行为，则直接使用方法的校验后入口地址，否则仍使用方法的普通入口地址。

由于大多数方法在入口处是否需要校验取决于运行时类加载的情况，因此全局立即数表中每一个方法需要同时记录普通入口地址和校验后入口地址。为了在立即数表中存放这些入口地址，需要建立一种机制，使得给定一个 Java 方法，能够确定翻译后代码入口地址在立即数表中的位置，即建立从被翻译 Java 方法到立即数表中存放位置的一对一映射。

本文提出的查表方案是为每个被翻译的 Java 方法分配一个 id。该 id 在 Java 方法被翻译完成时唯一确定。id 为 1 的方法存在立即数表本地方法入口地址区的第 1 项， id 为 2 的方法存在本地方法入口地址区的第 2 项，依次类推，id 为 n 的方法存在本地方法入口地址区的第 n 项。这样每调用一个方法，只需从方法的描述信息中取出方法的 id，就能确定该方法的入口地址在立即数表中存放的位置。

根据 id 查找方法入口地址的过程如图所示：

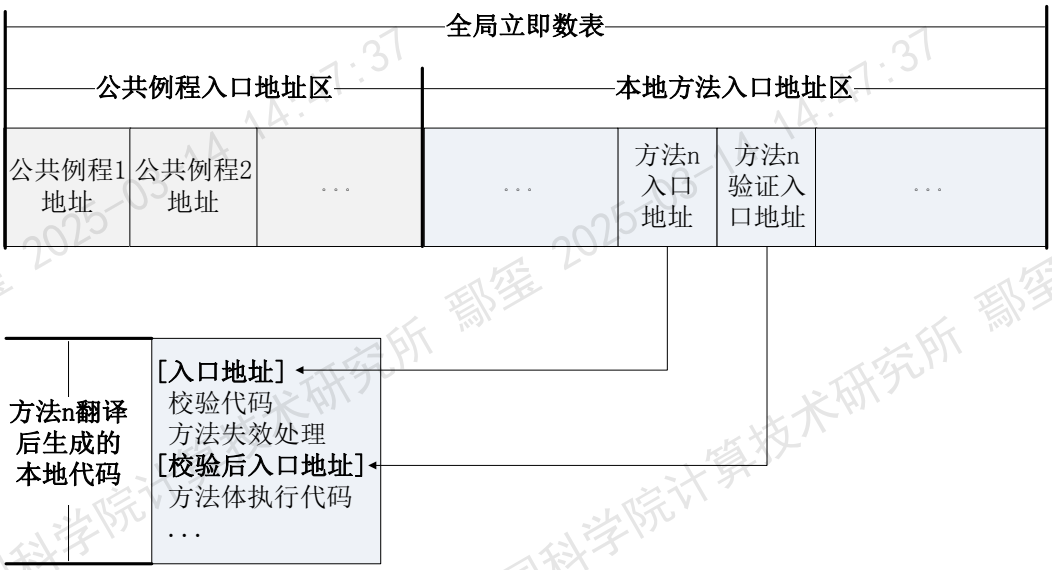

图 3-5 方法入口地址查找方式

找到相应表项后，将表项中的 64 位入口地址加载到目标寄存器中通过访存指令完成。MIPS64 下使用 ld 指令，格式为：ld 目标寄存器 偏移(基址寄存器)。

需要从 MIPS 的 32 个通用寄存器中选择一个空闲寄存器用作基址寄存器。每个立即数表项占用 64 位，即 8 字节，而每个方法的两个入口地址在立即数表中占用两项，共计 16 字节，因此在表中相应的偏移量则为 16 * id。

因为访存指令寻址的偏移是有符号的，因此基址寄存器指向全局立即数表的中点偏移量计算最为方便。记虚拟机公共例程入口地址为 N，立即数表总长度为 L，则编号为 Id 的被翻译方法普通入口地址按如下公式计算：

$$\mathrm{Offset} = 8 * \mathrm{N} + 16 * \mathrm{Id} - \frac{\mathrm{L}}{2}$$

相应地，编号为 Id 的被翻译方法验证后入口地址为(Offset + 8)，即

$$\mathrm{Offset} = 8 * \mathrm{N} + 16 * \mathrm{Id} - \frac{\mathrm{L}}{2} + 8$$

3.3.2 动态运行控制流

虚拟机运行时，当被调方法被识别为热点方法，就会交给即时编译器进行编译。编译完成后方法的 id 确定，在立即数表中存放的位置就能够确定，典型的方法入口地址填入立即数表的过程如下图所示：

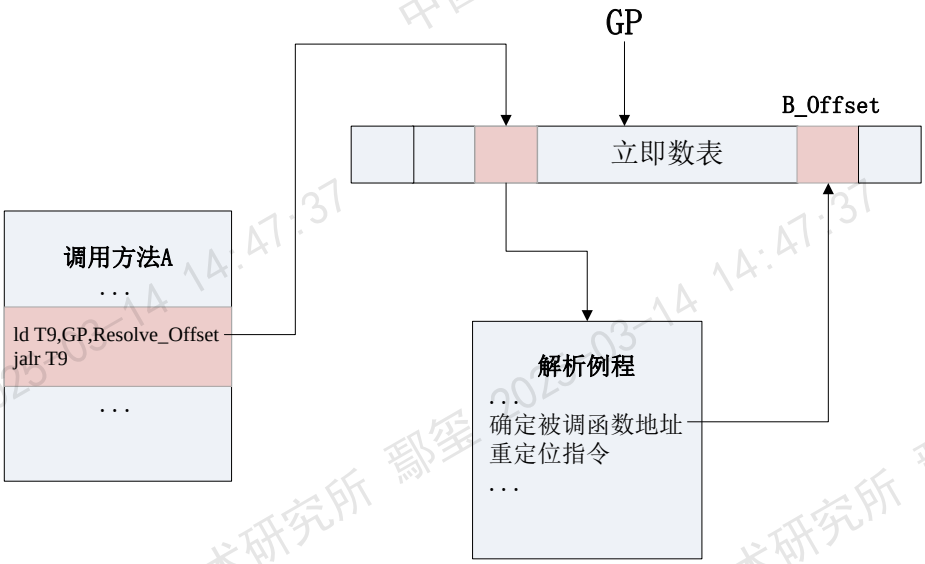

图 3-6 方法入口地址填入立即数表过程

方法 A 运行过程中需要调用方法 B，即时编译器在翻译方法 A 时会在调用方法 B 的位置生成一条访存指令 ld T9，GP，Resolve_Offset 和一条跳转指令 jalr T9。其中 T9 为 MIPS 提供的通用寄存器，通常用于保存目标跳转地址。GP 寄存器用于保存全局立即数表的基地址(指向立即数表的中点)，Resolve_Offset 是一个特殊的偏移量，指向立即数表中的一个特殊位置，立即数表中在相应的偏移处

会存放虚拟机提供的公共解析例程的地址。解析例程用于解析类继承关系并计算出被调方法的实际入口地址。

方法 A 的编译后代码第一次执行时，会以 GP 寄存器为基址，Resolve_Offset 为偏移，将立即数表中的解析例程入口地址加载到 T9 寄存器中，然后跳转到 T9 寄存器保存的地址，也就是解析例程的入口处。

进入解析例程，虚拟机会根据当前运行时的栈帧中的信息，以及类的继承关系，分析并确定当前实际要调用的方法 B。如果此时方法 B 尚未被翻译，则退回解释模式继续执行；如果方法 B 已经被翻译，那么方法 B 的入口地址已经被填入立即数表中，根据方法 B 的 id 能够确定入口地址在立即数表中存放的偏移 B_Offset。随后会进行一步称为重定位的操作，将调用方法 A 的翻译后代码中调用方法 B 的 ld 指令中的偏移由 Resolve_Offset 改写为 B_Offset。如下图所示：

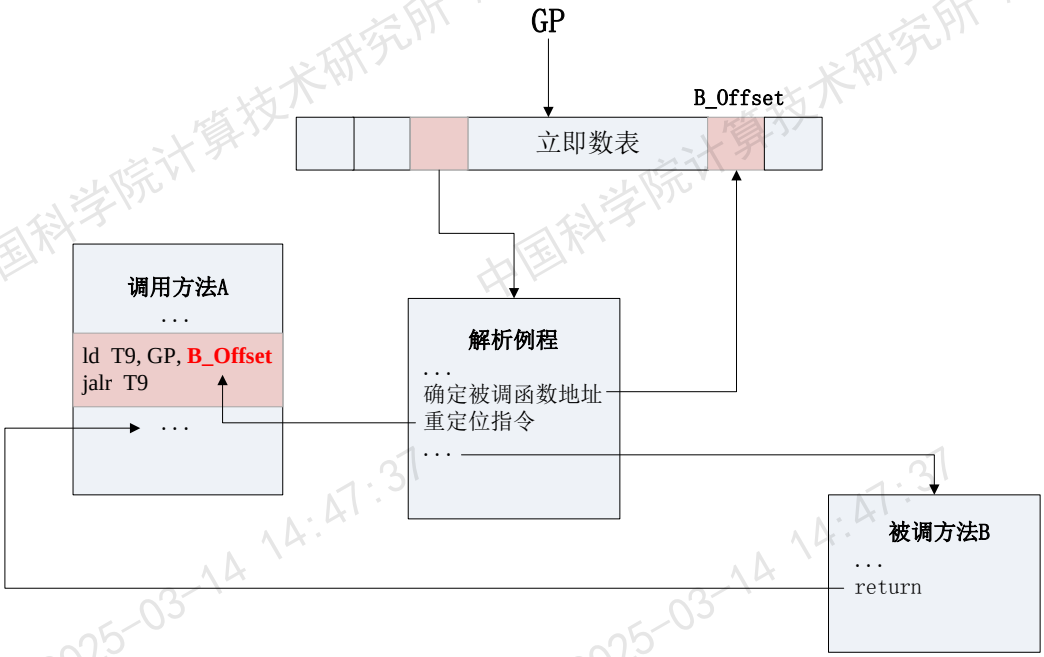

图 3-7 方法首次被调用执行过程

完成重定位操作后，虚拟机会从解析例程直接跳转到被调方法 B 中执行。执行完方法 B 后会直接返回调用方法 A 中 B 的调用点之后继续执行。MIPS 中通过寄存器 RA 保存方法的返回地址。从方法 B 到方法 A 的正确返回过程是通过借助保存和恢复 RA 寄存器的值实现的。

当方法 A 后续再被执行，执行情况将同第一次有所不同。当执行到调用方法 B 的位置时。首先通过 ld T9，GP，B_Offset 指令，将方法 B 的入口地址从立即数表的 B_Offset 处取出加载到寄存器 T9 中，然后通过 jalr T9 指令跳转到 T9 寄存器中保存的地址处，也就是方法 B 的入口地址处。从而开始执行被调方法 B。当方法 B 执行完毕后，能够返回方法 A 对 B 的调用处继续执行。

执行过程如下图所示：

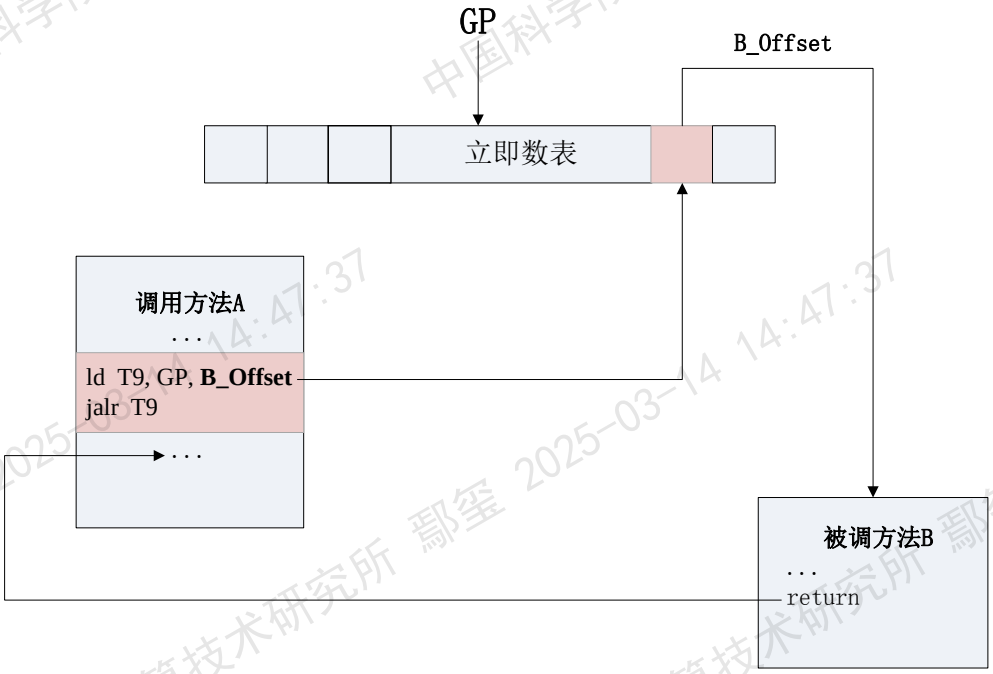

图 3-8 方法后续被调用执行过程

按照上述方案除去第一次调用进行了解析和重定位操作，后续的调用过程均只需要一条访存指令，即可将目标方法的跳转目标地址加载到寄存器中。相比现有的虚拟机 MIPS 实现，使用 6 条指令进行加载的方式，显著减少了指令条数，提升了翻译后代码质量。

3.4 算法实现

本方案基于 64 位龙芯 JDK 6 Client 版实现(基于开源 OpenJDK 6)。Java 虚拟机内部的公共例程入口地址列表如下：

表 3- 2 虚拟机内部公共例程入口介绍

公共例程入口名称	作用
wrong_method_blob	当前执行的被翻译方法出错时的处理例程
ic_miss_blob	虚拟机内部有一套称为 Inline Cache 的机制，用于将方法的代码内联，由于虚方法的多态行为可能导致 Inline Cache 失效，失效时由该例程处理。
resolve_opt_virtual_call_blob	当 Java 方法所属的类已被静态加载且为 final 方法时，该方法可被静态绑定，当

	作 static 方法处理，由该例程负责完成解析工作。
resolve_virtual_call_blob	一般 Java 方法的通用解析例程。
resolve_static_call_blob	静态 Java 方法的通用解析例程。
polling_page_safepoint_handler	虚拟机为支持 Java 规范中的异常处理，运行过程中会插入大量安全点，称为 safepoint，记录运行时信息以便于异常出现时终止运行并进行回溯，此例程用于对安全点的处理。
polling_page_return_handler	同 safepoint_handler_blob 类似，用于对异常处理提供支持的例程。
throw_range_check_failed	抛出 ArrayIndexOutOfBoundsException 的处理例程，用于虚拟机对数组越界情形的处理。
throw_index_exception	虚拟机抛出 IndexOutOfBoundsException 的处理例程。
throw_div0_exception	虚拟机抛出除 0 操作异常的处理例程。
throw_null_pointer_exception	虚拟机排除空指针异常时的处理例程。
throw_incompatible_ class_change	虚拟机有不兼容类型转换时的处理例程。
counter_overflow	虚拟机内部计数器溢出时的处理例程。

上述公共例程入口地址全部存放在全局立即数表的公共例程入口地址区中。

综上，全局立即数表 immTable 实现如下，64KB 空间容纳 8K 个立即数。分为公共例程入口地址区和翻译后方法入口地址区。公共例程入口地址区包含 13 项，每个表项占 8Byte，分别存放表 3-1 所述的 13 项公共例程的入口地址。立即数表中其余部分存放翻译后方法入口地址区，包含接近 4K 项。每个表项占 16 Byte，8Byte 用于保存方法入口地址，8Byte 用于保存方法验证后入口地址。翻译后方法由 nmethod 数据结构存储，为每个 nmthod 分配一个 immTable_id 记录在立即数表中占用的表项。

MIPS 寄存器命名约定及用法如下[5]：

表 3-3 MIPS 寄存器约定及其用法

寄存器编号	助记符	默认用途
0	ZERO	总是返回 0。
1	AT	为汇编保留。
2, 3	V0, V1	子程序返回值。
4 – 7	A0 - A3	用于保存子程序的前几个参数。
8 – 15	T0 - T7	临时寄存器，子程序使用无需存储和恢复。
16 - 23	S0 - S7	子程序寄存器变量，使用这些寄存器的子程序必须在进入前存储旧值并在退出前恢复。
24, 25	T8 - T9	临时寄存器，子程序使用无需存储和恢复。
26, 27	K0, K1	为中断/陷入处理器保留，值也可以改变。
28	GP	全局指针，某些试试系统用于为某些静态变量或外部变量提供简单的访问方式。
29	SP	堆栈指针。
30	S8/FP	第 9 个寄存器变量;子程序用它作为帧指针。
31	RA	子程序返回地址。

Java 运行时环境中，GP 寄存器是没有被使用的空闲寄存器，可以用作全局立即数表的基址寄存器。但是在 C/C++动态库运行时，GP 寄存器被用作静态数据区的全局指针。因此，在通过 Java 提供的接口调用 C/C++实现的本地方法时，需要保存 GP 寄存器的值，而当 C/C++方法执行完回到本地方法时，需要回复 GP 寄存器的值。这一工作在 Java 运行环境和 C/C++运行环境之间进行切换时进行。

通过 MIPS 访存指令 ld 从立即数表中读取方法入口地址。ld 指令提供 16 位表示偏移，寻址空间为 2^{16},即 L = 64K。因此偏移量的计算公式为

$$\mathrm{Offset} = 8 * 13 + 16 * \mathrm{Id} - 32000$$

整个方案对虚拟机的修改主要包括三部分，首先是全局立即数表 immTable 数据结构的实现及对立即数表增删、查找维护的操作实现。其次是对基址寄存器 GP 的初始化以及环境切换时保存恢复操作的实现。第三是即时编译器中同方法调用和解析有关的所有翻译规则均需要针对新的访存加载方式做相应的修改，一些特殊情况下对翻译后指令的处理同原来的加载方案处理会所不同，如方法进行逆优化和对方法调用进行重定位等。

3.5 本章小结

本章对 MIPS 体系结构下 64 位立即数加载的原理和来源进行了分析。随后提出一种虚拟机翻译执行过程中，通过在增加全局数据结构立即数表，并且在翻译后代码中使用固定寄存器为基址，根据特定偏移量加载虚拟机翻译后方法和公共例程入口地址的方法。

本文实现的优化方案主要包括如下三个关键点：

1. 对虚拟机静态数据结构进行了改造，增设了全局立即数表，用以保存 Java 方法即使编译后生成的本地代码的入口地址以及虚拟机内部公共例程的入口地址。

2. 在虚拟机运行过程中使用固定寄存器 GP 存储立即数表的中点，用作立即数表的基址寄存器，并在和 C/C++运行环境切换时对 GP 寄存器的值进行保存和恢复。

3. 修改即时编译器对方法调用的翻译规则，以固定寄存器 GP 的值为基地址，方法入口地址在立即数区的存放的位置为偏移，从立即数表中以访存方式直接读取到寄存器中。

本文提出的优化方案，相比传统将入口地址作为操作数通过多条立即数加载指令和移位指令加载到寄存器中的方式，在保证功能正确的前提下，减少了加载 Java 法入口地址和公共例程入口地址的指令条数，减少了翻译后代码占用空间，同时提高了翻译后代码的质量和执行效率。

第四章　　虚拟机运行时类库优化

Java 运行时环境(Java Runtime Environment, JRE) 是运行 Java 程序必须的环境的集合，主要由 JVM 标准实现及 Java 核心类库构成。Java 的应用程序接口(API) 以包的形式来组织，每个包提供了大量的相关类、接口和异常处理类，这些包的集合构成了 Java 类库。在 Java 程序运行过程中会大量调用类库提供的方法，虚拟机执行类库方法的效率对应用性能的影响至关重要。

4.1 研究背景

以 OpenJDK 为例，OpenJDK6 最新版本核心类库大约 42 万行代码，包含 32 万行纯 Java 代码，10 万行 C 代码，提供了近 8000 个类文件。OpenJDK 8 类库规模进一步增长到约 72 万行，其中 Java 代码增长到 62 万行，包含大约 9000 个类文件。类库编译后生成的.字节码文件会被打包集成在 Java 运行时环境中。

如 1.2 所述，字节码文件(.class)作为 Java 虚拟机执行的目标文件，描述了类的各种信息。JVM 需要将这些数据由 Class 文件加载到内存中，并完成校验、解析以及初始化操作，最终得到的数据才能由 JVM 直接操作。Java 虚拟机提供的核心类库规模庞大，类文件繁多，不可能全部加载到虚拟机内存中，因此只能够在运行时通过类加载器按需要加载完成[24]。

现代虚拟机经过多年的优化研究，Java 核心类库中最通用的一部分已经进行过深层次的优化，但对大量外围类的优化空间的挖掘尚未达到尽头。由于类库规模庞大，对每一个类都进行深度优化代价过高。一种可行的方法是从类库中找出少部分最影响系统性能的类重点投入优化。

基于上述分析，本文提出一种基于类库方法使用频度度量的热点类优化方法。

4.2 类库频度度量

虚拟机在运行过程中提供了剖析功能，能够对每个被调方法的调用次数进行计数，并提供了一组参数对虚拟机各执行模式执行时间分成进行统计。SpecJVM 2008 是用于评测 Java 虚拟机性能的标准测试集，每项用例都代表了某种典型的实际应用场景，本文以 SpecJVM 2008 作为测试用例，对各测试项运行过程中各模式的执行时间分成和类库方法调用频度进行了统计和度量。

以 SpecJVM 2008 的 xml.transform 一项为例，虚拟机运行过程中执行模式的

时间分成如下所示：

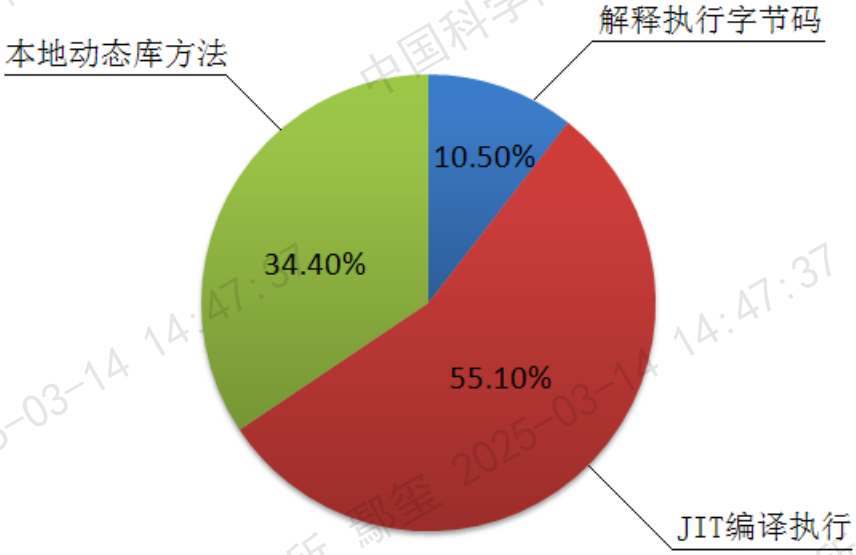

图 4-1 虚拟机运行 xml.transform 执行模式时间分成

可见，解释执行占总时间的比例较少，只有 10.5%。大部分时间花在执行 JIT 翻译后代码，另有三分之一的时间用于运行本地动态库方法，即通过 JNI 调用执行 native 方法。进一步，对具体执行的方法占用的时间比例进行统计，结果如下图所示：

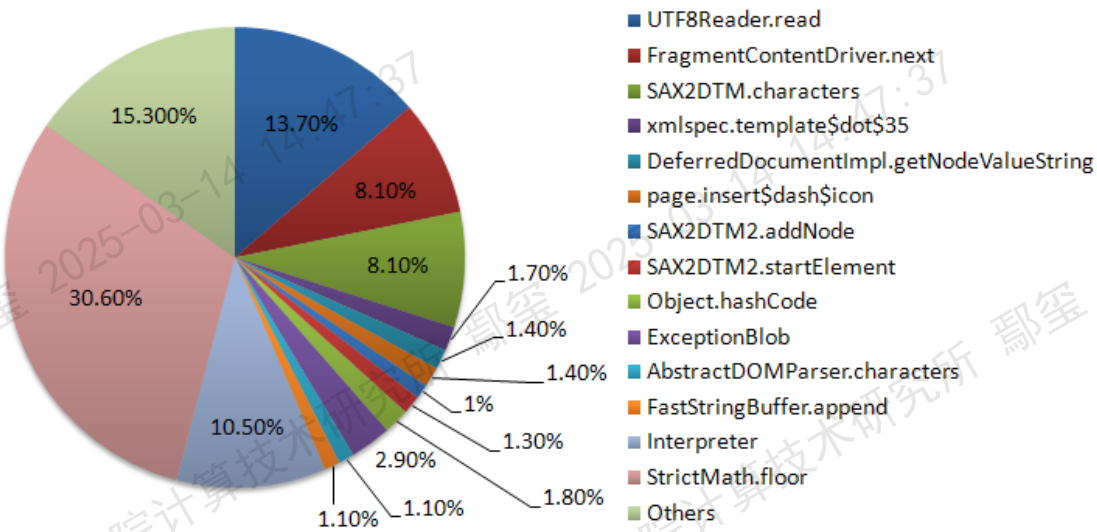

图 4-2 虚拟机运行 xml.transform 方法调用频度统计

以 xml.transform 为例，虚拟机实际执行过程中有 30.6%的时间在执行 java.lang.StrictMath.floor()方法。另外有 13.7%的时间用于执行 UTF8Reader.read()方法，有 8.1%时间用于执行 Object.hashCode()方法，均占有相当大的比例。

针对 SpecJVM 的其他测试用例的类库使用频度也进行统计和度量，统计结果显示，虚拟机运行过程中类库中各方法执行的频度不是均匀分布的，有一些方法的执行频度明显比其他方法更高，存在一定的时间局部性,。在一些典型场景下，小部分类库方法占用了大部分虚拟机执行时间。

针对这种现象，可以将类库中的被调方法按使用频度从高到低排序，然后重点优化高频的调用方法。高频度调用的方法既有可能是 Java 方法，也可能是本地方法。本文分别针对这两种情况提出两种相应的优化方案。

4.3 Intrinsics 机制优化

4.3.1 Intrinsics 机制工作原理

Intrinsics 机制是 Java 虚拟机提供的一种在虚拟机内部提前内置 C/汇编语言函数的方法。现代主流虚拟机，如 Hotspot，会提前定义一些 Intrinsics 方法，从而可以定义自己独有的函数，并可根据不同的架构使用不同的指令集。常用的数学函数如 Math.sin，Math.cos 等方法都已经被内置为 Intrisics 方法，通过汇编指令实现。因为手写汇编的执行效率远高于 Java 方法的执行效率，因而对于高频执行的方法可以通过这种方式提升执行效率。

利用 Intrinsics 机制优化的工作原理如下图所示：

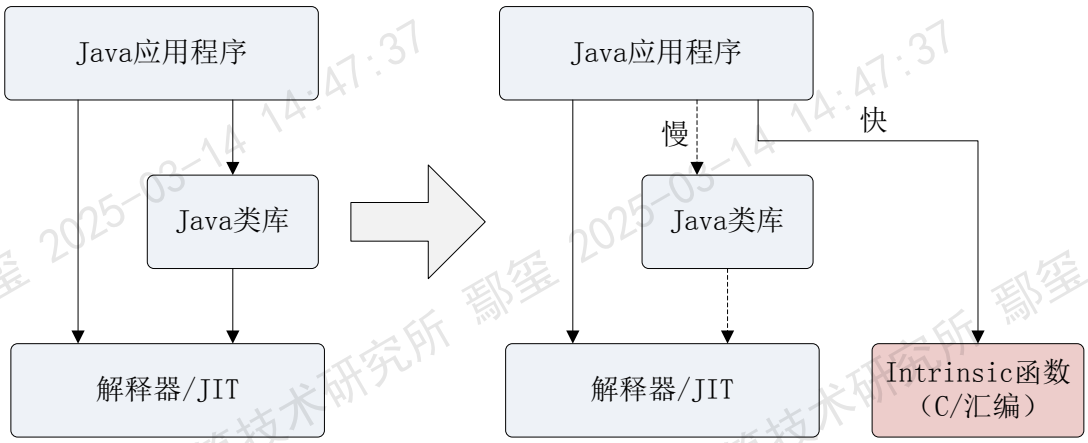

图 4-3 Intrinsics 机制优化工作原理

原方案中，Java 类库中的方法与 Java 应用程序中的方法均通过解释器解释执行，或者通过 JIT 编译器编译执行。即使是本地方法，通过 Java 提供的接口调用也需要进行从 Java 运行环境到 C/C++运行环境的切换，对于体积较小的方法，切换的代价占总执行开销的比例过大，执行效率仍然较低。

根据对类库频度度量的统计结果，可将高频度方法中规模较小的方法，改写为内置在虚拟机中的 Intrinsics 方法。带来的优化效果来自两方面，首先是执行效率上手写汇编高于 Java 方法或 C 方法。其次，即使原方法已经是 C/C++实现的本地方法。优化后仍然使用 C 语言实现该 Intrinsics 方法，因为节省了运行环境切换时栈帧格式转换的开销，仍然能够带来执行效率的提高。

4.3.2 利用 Intrinsics 机制实现优化

Hotspot 虚拟机中会为每个方法都会定义一个 instrinisics id, 这个 id 可以用于区分 java 里自己定义的 lib 类的方法和用户自己定义的 java 类的方法，用户自己写的类会用 vmIntrinsics::_none 来表示。对每个方法进行初始化时，会该对方法的 instrinisics id 也进行初始化。

Java 虚拟机中自己定义的 lib 方法通过一张表来记录，每个表项包括方法的名称、方法的 intrinsics id 以及方法的参数返回值等特征信息。最新版本的虚拟机是通过维护 vmSymbols 中的宏定义来实现的。例如：do_intrinsic(_hashCode, java_lang_Object, hashCode_name, void_int_signature, F_R) 将 java.lang.Object 的 hashCode 方法定义为 intrinsics 方法，intrinsics id 为_hashCode。

被声明为 Intrinsics 的方法将会直接被内置在虚拟机中，而不需要类加载、解释执行和即时编译这些过程。实现该功能的核心组件是虚拟机内部的 CallGenerator。CallGenerator 根据当前方法的 instrinsic_id 决定以何种方式运行。Java 虚拟机提供了多种 CallGenerator，对 Java 自定义的 lib 库中的方法，JVM 使用 LibraryIntrinsic 作为 CallGenerator。

CallGenerator 的核心工作是生成目标方法调用的本地代码，在所有 lib 方法的实现被内联集成在 LibraryCallKit 中，通过 CallGenerator 的 generate 方法调用。最新版本的 JDK 中对内置方法的实现位于 libaray_call.cpp 中。

以方法 java.lang.StrictMath.floor 方法为例，该方法是一个 native 方法，通过 JNI 机制调用，方法的功能是向下取整。反汇编 java.lang.StrictMath.floor 方法所在的动态库 libjava.so，截取 StrictMath.floor()的相关指令序列，如图 4-4 所示。

观察反汇编结果发现 StrictMath.floor()是一个规模很小的方法，指令条数很少，该方法实际执行过程中方法体本身指令序列的执行时间占的比重很少，大量时间消耗在参数传递、从 Java 运行环境到本地动态库运行环境的 ABI 转换、函数内部的 mtc1 等操作，而该方法在 xml.transform 这样的测试用例中执行频度竟然达到的 30%。像这样的高频度、低复杂度类库方法，非常适合内置于虚拟机中。

```
0000dd60 <Java_java_lang_StrictMath_floor>:
    dd60:       3c1c0004        lui     gp,0x4
    dd64:       279ca6f0        addiu   gp,gp,-22800
    dd68:       0399e021        addu    gp,gp,t9
    dd6c:       44866000        mtc1    a2,$f12
    dd70:       8f99807c        lw      t9,-32644(gp)
    dd74:       44876800        mtc1    a3,$f13
    dd78:       03200008        jr      t9
    dd7c:       00000000        nop
```

图 4-4 反汇编 StrictMath.floor()方法结果

在 vmSymbols 中为 java.lang.StrictMath.floor()增加一项，标记为 intrinsics 方法：do_intrinsic(_floor, java_lang_StrictMath, floor_name, double_double_signature, F_SN)。同时将方法的具体实现从 java.lang 包中改为集成在虚拟机提供的内部类 LibraryCallKit 中。这样一来就以极小的实现代价改变了虚拟机对类库方法 StrictMath.floor()的调用机制，提升了执行效率。

4.4 JNI 调用优化

4.4.1 JNI 机制介绍

Java 本地接口[25](Java Native Interface, JNI)标准是 Java 平台的一部分，为 Java 程序与 C/C++编写的代码进行交互提供了支持。Java 本地接口的工作结构如下：

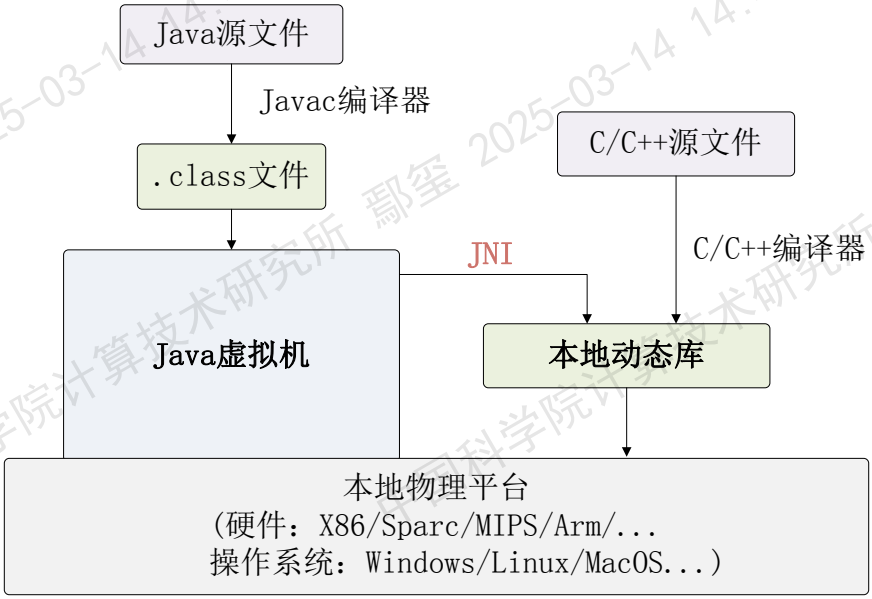

图 4-5 Java 本地接口工作结构

Java 源程序通过 Javac 编译工具生成的字节码文件能够直接被 Java 虚拟机执行。Java 程序中需要由 C/C++实现的方法用 native 关键字声明，在 Java 中不提供方法实现。native 方法的实现编写在另外的 C/C++源文件中，实现遵循 JNI 约定的规范，通过 C/C++编译器编译为本地动态库，能够直接在本地物理平台上运行。虚拟机在运行 Java 程序过程中需要使用本地方法时，通过 JNI 接口进行调用，执行本地动态库中的函数，运行结束后返回 Java 虚拟机运行环境继续运行。

以 jnitest.java 程序示例，JNI 使用方法如下所示：

```
[jnitest.java]
 public class jnitest {
   static
  {
    System.loadLibrary("jnitest");
  }
   public static native int add(int a, int b);
   public static void main(String[] args) {
       int n = 0;
       for (int i = 0; i < 40000000; i++)
        n += add(i, i);
       System.out.println(n);
  }
}
```

①

```
[jnitest.c]
 #include "jnitest.h"
 #include <stdio.h>
 #include <string.h>
 JNIEXPORT jint JNICALL Java_jnitest_add
   (JNIEnv *env, jclass cls, jint a, jint b)
{
   return a + b;
}
```

③

```
$ gcc jnitest.c -c  -fPIC --shared
    -I /java/include/
    -I /java/include/linux/
$ gcc jnitest.o -o libjnitest.so
    --shared
```

④

```
$ javac jnitest.java
$ javah jnitest
```

②

```
$ export LD_LIBRARY_PATH=.
$ java jnitest
```

⑤

图 4-6 JNI 使用方法示例

第一步，在 jnitest 类中声明一个 native 方法 add，用于完成两个数的加法操作，但不提供方法体；通过 System 类的 loadlibrary 方法加载动态库 libjnitest.so，该库将在后面的步骤中生成；编写好测试用例。第二步，直接将 jnitest.java 编译为 jnitest.class 文件。借助 javah 工具，能够为 jnitest 中的 native 方法生成一个符合 JNI 调用规范的头文件 jnitest.h。第三步，按照 jnitest.h 中的声明编写 C/C++源文件 jnitest.c，为本地方法 add 提供方法体实现。第四步，使用 gcc/g++编译器将 jnitest.c 编译为本地动态库 libjnitest.so。第五步，运行 jnitest，两数相加功能将能够通过调用本地方法 add 正确实现，同直接在 Java 源程序中实现 add 是功能等价的。

利用 JNI 机制，可以将高频度调用的 Java 方法本地化，即改写为本地方法。因为只是针对少量方法进行改造，对平台相关性的影响并不明显。同时，本地代码无需解释执行和动态编译的过程，因此能够提升方法的执行效率，从而改善系统性能。

4.4.2 利用 JNI 机制实现优化

利用 JNI 调用本地化 Java 方法的过程中需要解决 JNI 参数传递和本地方法中对 Java 层的数据操作访问的问题。Java 虚拟机负责将 Java 栈帧转换为 C/C++本地栈帧。当 JVM 进入 JNI 调用，除了函数本身的参数外，还会多出另外两个参数：JNIEnv 指针和 jobject 指针。

JNIEnv 指针是由虚拟机创建的，用于本地实现的 C/C++方法操纵 Java 运行时环境中的数据，包括各种 Java 类和 Java 方法。JNI 方法接受的第二个参数是 Java 对象：jobject，可以在 JNI 中操作这个 jobject 进而操作 Java 对象提供的变量和方法。

本地方法访问 Java 层类和对象数据需要借助 JNI 提供的类和对象的操作函数完成。常用的方法为 FindClass 和 GetObjectClass 方法，函数原型如下：

表 4-1 JNI 提供的类和对象的操作函数

函数原型	介绍
jclass FindClass(const char* name);	C++中根据类名查找 Java 类信息
jclass GetObjectClass(jobject obj);	C++中返回对象所属的 Java 类
jclass (*FindClass)(JNIEnv*, const char*);	C 中根据类名查找 Java 类信息
jclass (*GetObjectClass)(JNIEnv*, jobject);	C 中返回对象所属的 Java 类

在本地方法中获取类和对象信息后就能够使用 JNI 提供的操作域和方法的函数操作类和对象的域，函数原型如下：

表 4-2 JNI 提供的对域和方法的操作函数

功能	函数原型
访问对象的域	GetField, Get<Type>Field, Set<Type>Field
调用实例方法	GetMethodID, Call<Type>Method, CallNonvirtual<Type>Method
访问静态域	GetStaticFieldID, GetStatic<Type>FieldID, SetStatic<Type>FieldID
调用静态方法	GetStaticMethodID, GetStatic<Type>Method

利用 JNI 提供的接口方法，就能够实现 C/C++与 Java 端的参数传递与数据访问，从而将原 Java 方法改写为等价的 C/C++实现。为了避免改写后的 C/C++方法又调用其他 Java 方法导致的运行环境切换开销，需要将待优化的目标方法

中调用的相关 Java 方法也都一并改写为 native 方法。因此，利用 JNI 机制改写类库方法的优化方案适用于方法体较大，方法体内对其他方法调用少的方法。另外，如果方法中存在循环，则能够进一步减少 C/C++栈帧到 Java 栈帧转换开销占总开销的比例，执行效率优化提升更明显。

以 UTF8Reader.read()方法为例，该方法的作用为从指定输入流按照 UTF-8 编码标准读取一个字符到指定缓冲区，在当前版本的 JDK 里该方法由 JAXP 组件提供，是一个纯 Java 方法。该方法在 xml.transfrom 测试用例中占总执行比例的 13.7%，在 xml.validation 等测试用例中也占有交大的执行时间分成，属于本文讨论的典型高频度执行的热点类库方法。

UTF8Reader.read 方法的核心操作是按照 UTF-8 编码的规范解析和读取字符，并进行校验，读取过程繁琐导致方法规模较大，不适合直接内联为虚拟机内部方法。但由于执行体执行指令条数较多，JNI 调用本身带来的参数传递和栈帧转换的开销占总时间的比例较小，因此通过将原方法 Java 实现改写为等价 C/C++实现，通过 JNI 调用将原方法转化为本地方法的手段，在保证功能等价的前提下提高执行效率。

4.5 本章小结

本章对 Java 虚拟机类库部分的发展与现状做了介绍，论述了运行时类库对虚拟机性能影响的重要性。随后以 SpecJVM 2008 为目标测试用例对类库使用频度进行了度量和统计。统计结果显示一些典型应用场景下，类库中方法的使用频度分布不均，存在少量高频度执行的热点方法。

针对高频度热点方法，借助 Java 虚拟机的现有机制支持，本章提出了两种优化方法，使用 Intrinsics 机制优化和使用 JNI 调用优化。两种方式均对原执行方法进行了一定程度的本地化，均能以 C 实现代替原有方法的 Java 实现。但两种机制的设计初衷与使用范围均不相同。

Intrinsic 方法与 JNI 调用两种机制的特点列表比较如下：

表 4-3 Intrinsic 方法与 JNI 调用比较

比较角度	Intrinsics 方法	JNI 调用
目标	提高解释器运行效率	提高 Java 语言的可扩展性
适用范围	Java 类库中少量热点方法	应用程序 Java 类库底层方法
C 代码设计者	虚拟机开发人员	应用程序开发人员

		Java 类库开发人员
C 代码位置	虚拟机内部	虚拟机外部
触发条件	执行 invoke 之类方法调用的字节码时，检查目标是否为虚拟机支持的 Intrinsics 方法	执行 invoke 之类字节码时，检查目标方法是否有 native 标志。

针对虚拟机运行时类库中高频执行的大型方法，更适合通过 JNI 调用本地化的方式进行优化，以较低的运行时切换代价换取执行效率的提升；针对高频执行的小型方法，更适合通过 Intrinsics 机制直接将方法内置于虚拟机中，以较小的空间开销换取性能提升。在下一章中，将对本章提出的类库优化方法进行效果测试与验证。

中国科学院计算技术研究所 鄢銮 2025-03-14 14:47:37

中国科学院计算技术研究所 鄢銎 2025-03-14 14:47:37

第五章　测试与验证

5.1 测试与验证平台介绍

5.1.1 龙芯处理器硬件平台介绍

本文研究工作采用龙芯软硬件平台进行测试与验证。

龙芯系列处理器芯片是中科院计算技术研究所微处理器研究中心自主研发的处理器系列芯片，采用 MIPS 体系结构，目前共有三个系列，其中 1、2 号为单核处理器，3 号为多核处理器。龙芯 3 号多核处理器，通过不断采用先进的工艺和设计方法，实现跨越式的发展。

2009 年龙芯第一款多核处理器 3A 处理器诞生，采用 65nm 工艺设计，片内集成了 4 个四发射 64 位处理器核和 4MB 二级 Cache，片上包含 4.25 亿个晶体管，主频达到 1GHz，功耗小于 15 瓦，峰值性能达到 160 亿次浮点运算/秒。随后研制的龙芯 3B 处理器同样采用 65nm 工艺设计和 1GHz 主频，并在片内集成了 8 个带向量扩展的超标量处理器核，片内双精度浮点运算速度达到 1280 亿次/秒，适用于高性能计算和数字信号处理等应用。另外研制的龙芯 3B1500 芯片采用 32nm 工艺设计，主频达到 1.5GHz，片内集成 8 个 64 位超标量向量处理器核，片内双精度浮点运算速度达到 1920 亿次/秒。

本文提出的优化方案均在龙芯 3 号系列芯片上进行测试与验证。

5.1.2 龙芯系统软件平台介绍

龙芯硬件平台上具有良好的软件生态环境支持。操作系统支持 Linux 的 Fedora 与 CentOS 发行版，目前最新支持的版本达到 Fedora 19 和 CentOS 6.4。龙芯 JDK 运行于操作系统之上，目前有龙芯 JDK 6 和龙芯 JDK 8 两个版本。JDK 之上能够运行各种 Java 应用，如 tomcat、eclipse 等等。本文研究选用 SpecJVM 2008 为测试用例，研究的测试验证平台配置如下表所示：

表 5-1 测试验证平台配置介绍

项目	名称	备注
硬件平台	龙芯 3A 双路	4 核，主频 1GHz，内存 4G
操作系统	CentOS 6.4	64 位操作系统

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37

虚拟机	龙芯 JDK 6 / 龙芯 JDK 8	JDK 6 采用 64 位 Client 版 JDK 8 采用 64 位 Server 版
测试用例	SpecJVM 2008	8 线程，混合模式运行

5.1.3 SpecJVM 2008 测试集介绍

本文采用 SpecJVM 2008 作为主要的测试工具。SpecJVM 2008 为标准性能评估机构(Standard Performance Evaluation Corporation, 简称 SPEC) 提供的 Java 虚拟机性能测试工具集，于 2008 年推出，它涵盖了大部分 Java 基础应用场景，使用了 Java 语言的核心功能，通过多个测试用例，从不同角度衡量 Java 虚拟机的性能。

SpecJVM 共包含 21 项测试用例，测试内容如下所示：

表 5- 2 SpecJVM 2008 测试集项目介绍

项目	测试内容介绍
compiler.compiler	普通 java 文件编译测试
compiler.sunflow	sunflow 源文件的编译测试
compress	使用修改过的 Lempel-Zivmethod (LZW)算法压缩测试
crypto.aes	AES/DES 加解密算法测试
crypto.rsa	RSA 加解密算法测试
crypto.signverify	使用 MD5withRSA, SHA1withRSA, SHA256withRSA 来签名，识别，并对数据进行加密操作测试
derby	数据库相关逻辑测试
。mpegaudio	测试 mpeg 音频解码
scimark.fft.large	测试快速傅立叶变换，使用 32M 大数据集，
scimark.lu.large	测试 LU 分解，使用 32M 大数据集
scimark.sor.large	测试 jacobi 逐次超松弛迭代法，使用 32M 大数据集
scimark.sparse.large	测试稀疏矩阵乘积，使用 32M 大数据集
scimark.fft.small	测试快速傅立叶变换，使用 512K 小数据集
scimark.lu.small	测试 LU 分解，使用 512KB 小数据集
scimark.sor.small	测试 jacobi 逐次超松弛迭代法，使用 512KB 小数据集，
scimark.sparse.small	测试稀疏矩阵乘积，使用 512KB 小数据集，
scimark.monte_carlo	测试蒙特卡罗算法
serial	测试通过 socket 传输 java 序列化对象到对端反序列化，

	测试数据来源于 JBoss
sunflow	测试图片渲染操作
xml.transform	测试将 XML 转换为其他类型文件
xml.validation	测试 XML schema 验证

测试分值单位为每分钟单项测例执行的操作次数(operations per minute，简称 ops/m)，分值越高代表性能越好。

5.2 64 位立即数加载优化测试

5.2.1 立即数加载优化即时编译代码效果对比

龙芯 JDK 6 基于 OpenJDK 6[26]，移植在 MIPS64 平台上。使用移植后的最新 64 位版本运行 Java 应用，使用虚拟机提供的+XX:PrintAssembly 参数打印动态翻译后生成的本地代码。以 java.util.HashMap::put 为例，截取翻译后代码片段如下：

```
128  b  java.util.HashMap::put (126 bytes)
Decoding compiled method 0x00000055652e2c50:
Code:
[Entry Point]
 ;; block B14 [0, 0]

 0x00000055652e2dc0: ld at, 0x8(t0)
 0x00000055652e2dc4: beq at, t1, 0x00000055652e2de4
 0x00000055652e2dc8: nop
 0x00000055652e2dcc: lui t9, 0x55                    <-- li48指令加载立即数
 0x00000055652e2dd0: ori t9, t9, 0x64fb
 0x00000055652e2dd4: dsll t9, t9, 16
 0x00000055652e2dd8: ori t9, t9, 0x23c0
 0x00000055652e2ddc: jr t9
 ...
 0x00000055652e2e60: lui t9, 0x55     ;  {virtual_call}      <-- li48指令加载立即数
 0x00000055652e2e64: ori t9, t9, 0x64fb
 0x00000055652e2e68: dsll t9, t9, 16
 0x00000055652e2e6c: ori t9, t9, 0x2d40
 0x00000055652e2e70: jalr t9
 0x00000055652e2e74: nop          ; OopMap{[72]=Oop [88]=Oop [80]=Oop off=184}
                                  ;*invokevirtual hashCode
 ...
```

图 5-1 实例 java.util.HashMap::put 方法翻译后代码

方法的入口处以[Entry Point]作为标识，随后是一段 64 位立即数加载指令序

列，0x0000005564fb23c0 实际上是虚拟机公共例程 ic_miss_blob 的入口地址，该例程用于对虚方法实际调用对象发生变化的情况进行处理。

java.util.HashMap::put 方法执行过程中调用了虚方法 hashcode，具体是哪一个对象的 hashcode 在运行时刻才能确定。在调用处也出现了一段 64 位立即数加载序列。这段序列加载的地址 0x0000005564fb2d40 正是跳转的目标方法地址。首次填入的是公共解析例程的方法，后续在实际执行时会将操作数重定位为实际调用对象的 hashcode 方法的入口地址。

基于龙芯 JDK6 对第三章所述立即数加载方案进行了实现，优化后运行相同 Java 应用，仍截取 java.util.HashMap::put 翻译后代码如下：

```
128  b   java.util.HashMap::put (126 bytes)
Decoding compiled method 0x000000555d2e8a90:
Code:
[Entry Point]
 ;;  block B14  [0, 0]

  0x000000555d2e8c00: ld at, 0x8(t0)
  0x000000555d2e8c04: beq at, t1, 0x000000555d2e8c18
  0x000000555d2e8c08: nop
  0x000000555d2e8c0c: ld t9, 0x10(gp)            <-- li48指令替换为ld指令
  0x000000555d2e8c10: jr t9
  0x000000555d2e8c14: nop
  0x000000555d2e8c18: nop
  0x000000555d2e8c1c: nop
  ...
  0x000000555d2e8c80: ld t9, 0x30(gp)   ;   {virtual_call}
  0x000000555d2e8c84: jalr t9
  0x000000555d2e8c88: nop            ; OopMap{[72]=Oop  [88]=Oop  [80]=Oop  off=140}
                                     ;*invokevirtual hashCode
  ...
```

图 5-2 优化后 java.util.HashMap::put 方法翻译后代码

可见，方法入口处之后的立即数加载指令序列已经消失，取而代之的是一条访存指令 ld t9, 0x10(gp)。其中 t9 为跳转目标寄存器地址，gp 为全局立即数表基址寄存器，偏移量为 0x10，即十进制 16。立即数表中每项 16 字节，因此 0x10 对应立即数表中的第二项。如 3.4 算法所述，第二项存放的正是 ic_miss_blob 入口地址，同优化前的功能是等价的。

在 java.util.HashMap::put 方法对 hashCode 方法的调用处指令也发生了变化。原来的立即数指令加载序列被替换为访存指令 ld t9, 0x30(gp)。这里的偏移对应立即数表中的第四项，对应的是虚方法调用的公共解析例程入口地址，说明当前方法的翻译后代码还是初次被执行，初次运行完成后将会修改本条访存指令的偏

移量，修改为实际调用的 hashCode 方法的入口地址在立即数表中存放的位置偏移。这一过程同优化前的指令序列功能也是等价的。

对比优化前后翻译后代码，可以看出优化后即时编译器生成机器指令的条数明显减少，每一处方法或公共例程调用都能够节省 3~5 条指令。因而总的翻译后代码规模能够减少。以 SpecJVM 2008 标准例程为例，实际运行过程中进行统计，计算发现优化后的方法平均翻译规模比原来减少了 2.64%。

5.2.2 立即数加载优化性能对比测试

在龙芯 3A 双路开发板上，64 位 CentOS 6.4 操作系统，基于龙芯 JDK 6 进行优化。使用 Spec98、SpecJVM 2008 作为测试集。分配 1G 内存，8 线程运行，多次测试取平均值，优化前后测试分值如下：

表 5- 3 默认参数下立即数加载优化 SpecJVM 2008 测试分值

编号	项目	优化前	优化后	百分比
1	compiler.compiler	18.54	17.05	-8.04%
2	compiler.sunflow	8.99	8.8	-2.11%
3	compress	21.58	21.8	1.02%
4	crypto.aes	8.86	9.03	1.92%
5	crypto.rsa	19.3	19.32	0.10%
6	crypto.signverify	31.15	31.36	0.67%
7	derby	4.69	4.86	3.62%
8	mpegaudio	11.31	11.72	3.63%
9	scimark.fft.large	6.87	7	1.89%
10	scimark.lu.large	0.68	0.69	1.47%
11	scimark.sor.large	2.68	2.78	3.73%
12	scimark.sparse.large	1.63	1.69	3.68%
13	scimark.fft.small	28.69	29.03	1.19%
14	scimark.lu.small	10.33	10.61	2.71%
15	scimark.sor.small	19.35	19.52	0.88%
16	scimark.sparse.small	7.21	7.19	-0.28%
17	scimark.monte_carlo	10.33	10.3	-0.29%
18	serial	4.12	4.35	5.58%
19	sunflow	8.76	8.85	1.03%

20	xml.transform	15.42	16.51	7.07%
21	xml.validation	15.99	17.66	10.44%

测试结果显示，SpecJVM 2008 的 21 项测试用例中，除去 compiler.compiler 与 compiler.sunflow 两项以外，其余 19 项分值持平或提升。其中部分测试项如 serial, xml.transform, xml.validation 等有 5%~10%的显著提升。其余多数项目提升在 3~5%之间。整体性能平均提升 2.62%。

在龙芯 3A 双路开发板上，一个主板上有两个龙芯 CPU，共享物理内存。每一块物理内存都可以被两个 CPU 共享访问，但是速度不同。其中与 CPU 相连的一块物理内存称为本地内存，与另一块 CPU 相连的内存相对于当前 CPU 称为远程内存。测试数据表明，龙芯 CPU 访问远程内存的速度比访问本地内存的速度慢 50%。

虚拟机默认参数运行时，数据被分配在两块物理内存中，CPU 既有可能访问本地内存，也有可能访问远程内存。经过龙芯虚拟机团队优化，龙芯 JDK 提供了 UseNUMA 参数，用于将 CPU 与本地内存进行绑定，使得虚拟机的数据固定分配在本地内存上。这样虚拟机线程访问数据时只需要访问本地内存而不需要访问远程内存，从而提升访存效率。

UseNUMA 参数优化本身能够为虚拟机带来性能提升，对于本文提出的立即数加载方案，将大量立即数加载指令转化为访存指令，因此访存性能对立即数优化加载方案影响显著。使用 UseNUMA 参数能够进一步提升立即数加载方案的优化效果。

在龙芯 3A 双路开发板上，64 位 CentOS 6.4 操作系统，仍使用 Spec98、SpecJVM 2008 作为测试集。分配 1G 内存，8 线程运行，使用 UseNUMA 参数，多次测试取平均值，优化前后测试分值如下：

表 5- 4 UseNUMA 参数下立即数加载优化 SpecJVM 2008 测试分值

编号	项目	优化前	优化后	百分比
1	compiler.compiler	17.4	16.51	-5.11%
2	compiler.sunflow	7.79	7.65	-1.80%
3	compress	31.35	32.32	3.09%
4	crypto.aes	9.03	9.48	4.98%
5	crypto.rsa	19.32	19.75	2.23%
6	crypto.signverify	31.87	32.65	2.45%
7	derby	5.42	5.79	6.83%

8	mpegaudio	11.92	12.45	4.45%
9	scimark.fft.large	6.1	6.55	7.38%
10	scimark.lu.large	0.94	0.99	5.32%
11	scimark.sor.large	3.58	3.79	5.87%
12	scimark.sparse.large	2.05	2.16	6.40%
13	scimark.fft.small	27.6	28.65	3.80%
14	scimark.lu.small	17.55	17.83	1.60%
15	scimark.sor.small	22.27	24.02	7.86%
16	scimark.sparse.small	13.31	13.86	4.13%
17	scimark.monte_carlo	10.4	10.41	-0.10%
18	serial	4.65	4.86	4.52%
19	sunflow	9.03	9.46	4.76%
20	xml.transform	16.07	17.5	8.90%
21	xml.validation	16.11	17.78	10.37%

使用 UseNUMA 参数后，SpecJVM 2008 的 21 项测试用例中大多数项目性能优化幅度得到进一步提升。除了 compiler.compiler 与 compiler.sunflow 两项分值有降低外，其余多数项目性能提升在 5~8%之间。其中 xml.validation 由 16.11 提升至 17.78,提升 10.37%。xml.transform 由 16.07 提升至 17.78，提升 8.9%，优化效果显著。整体平均性能提升达到 5%，相比默认参数情况优化幅度得到进一步增加。

针对 compiler.compiler 以及 compiler.sunflow 两项测试用例，对虚拟机运行过程中立即数表的使用情况进行了分析。分析结果发现，compiler.compiler 与 compiler.sunflow 是整个 SpecJVM 2008 测试用例中虚拟机即时编译器翻译方法最多的两项。由于当前立即数优化加载方案在龙芯平台的实现使用 ld 指令完成立即数的加载。而 ld 指令留给表示偏移量的字段仅有 16 位，即一条 ld 指令能够寻址 2^{16} 字节空间，即 64KB。按照当前立即数表设计，每个表项装载一个方法的入口地址和验证后入口地址，即两个 64 位立即数的空间，共 16B。因此 64KB 能够装下 64KB/16B 个方法的入口地址，即 4000 个方法。当被翻译方法数量超过 4000 个时，立即数表就会填满，这时优化的立即数加载方案将不能再使用，采用的方法是退回解释模式继续解释执行。compiler.compiler，compiler.sunflow 两项，均是由于被翻译的方法过多达到立即数表上限而退回解释模式执行导致了方法分值的降低。

本文提出的 64 位立即数加载方案与原方案对比效果如下所示：

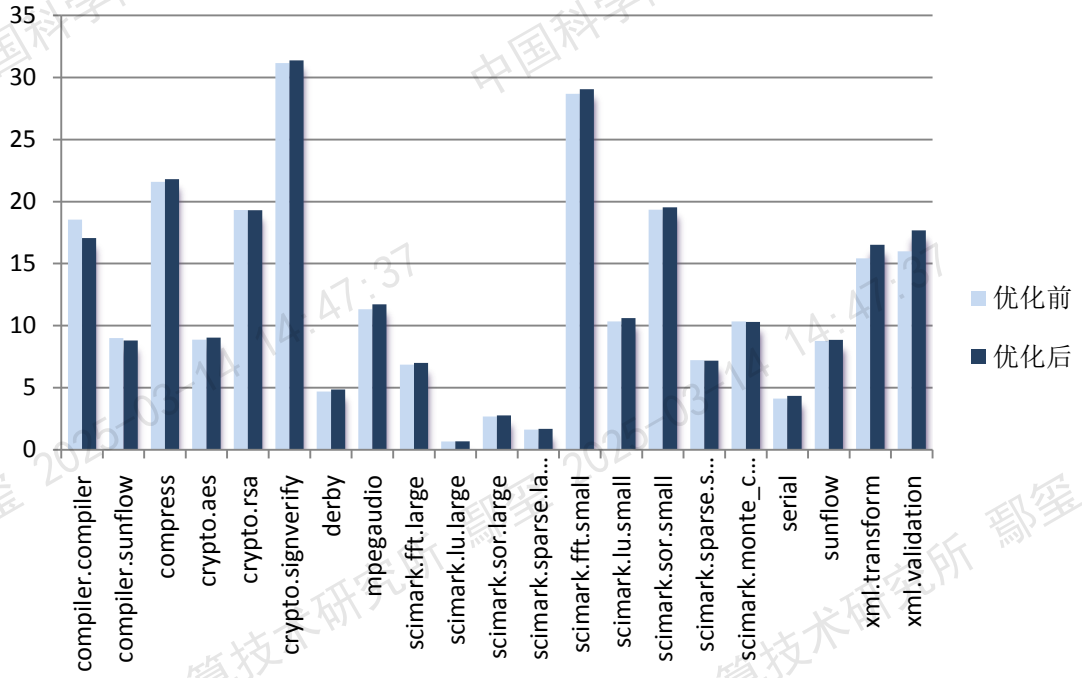

图 5-3 默认参数下立即数加载优化效果测试

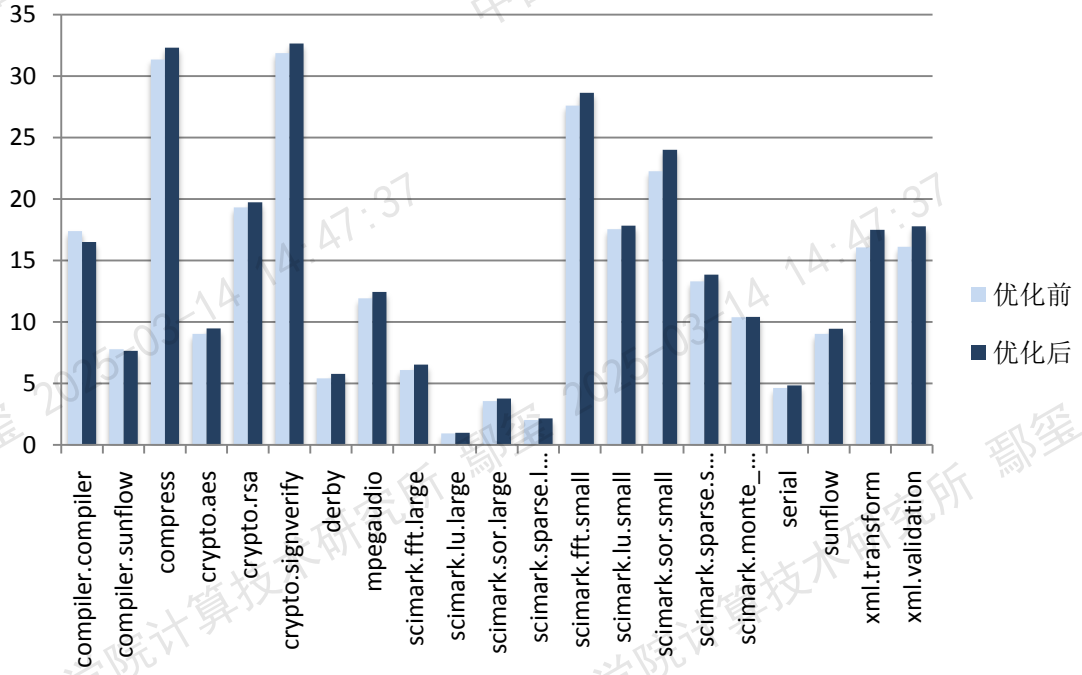

图 5-4 使用 UseNUMA 参数立即数加载优化效果测试

对于翻译的热点方法过多导致立即数表填满的情况，可以通过软件方法，重复利用立即数表空间增加立即数表利用率的方法解决；也可通过硬件方法，增设特殊硬件访存指令，增加访存的可寻址空间，从而解决立即数表的容量问题，获

得进一步的性能提升。

5.3 运行时类库优化测试

龙芯 JDK 8 基于 OpenJDK 8[27]，移植在 MIPS64 位平台上，相比 JDK 6 运行时类库更完善，功能更强大，更适合投入运行时类库优化。本文在 JDK 8 基础上针对 SpecJVM 2008 进行类库使用频度的度量，各项测试用例中占总执行时间超过 5%的类库方法判定为候选热点类库方法，投入热点类优化。对方法执行体较大、调用关系简单的 Java 方法利用 JNI 机制进行本地化，对方法执行体较小的 Java 方法或 native 方法利用 Intrinsics 机制直接内置于虚拟机中。

在龙芯 3A 双路开发板，64 位 CentOS 6.4 操作系统上，仍使用 SpecJVM 2008 作为测试集。分配 1G 内存，8 线程运行，使用 UseNUMA 参数，多次测试取平均值，优化前后测试分值如下：

表 5-5 运行时类库优化 SpecJVM 2008 测试分值

编号	项目	优化前	优化后	百分比
1	compiler.compiler	27.24	27.35	1.14%
2	compiler.sunflow	12.38	13.11	5.90%
3	compress	34.62	35.08	1.33%
4	crypto.aes	10.43	10.50	0.67%
5	crypto.rsa	25.67	25.64	-0.12%
6	crypto.signverify	34.67	35.38	2.05%
7	derby	14.68	14.71	0.20%
8	mpegaudio	15.42	16.02	3.89%
9	scimark.fft.large	4.43	4.94	11.51%
10	scimark.lu.large	0.86	0.87	1.16%
11	scimark.sor.large	4.5	4.55	1.11%
12	scimark.sparse.large	2.26	2.31	2.13%
13	scimark.fft.small	15.87	16.78	5.73%
14	scimark.lu.small	9.9	10.16	2.63%
15	scimark.sor.small	18.99	20.79	9.46%
16	scimark.sparse.small	12.47	13.33	6.93%
17	scimark.monte_carlo	13.9	15.2	9.35%
18	serial	5.25	5.25	0%
19	sunflow	11.64	11.69	0.43%
20	xml.transform	27.08	30.31	11.93%
21	xml.validation	21.14	21.75	7.99%

管优化涉及的 Java 方法占整个类库的比例极小，但由于投入优化热点类的

执行频度较高，因此优化后部分项目的性能有明显提升。仅 xml.transfrom 一项为例，仅仅通过对 java.lang.StrictMath.floor 方法和 UTF8Reader 类的优化，分值就由 27.08 提升到 30.31，提升 11.93%。其余项目中，xml.validation 由 21.14 提升至 21.75，提升 7.99%；数值计算方面，scimark.fft 也有显著提高。对于类库使用频度分布均匀的用例，如 compiler.compiler 等，优化前后分值持平。测试集平均性能提升 4.07%。

本文提出的运行时类库热点类优化方案与原方案对比效果如下所示：

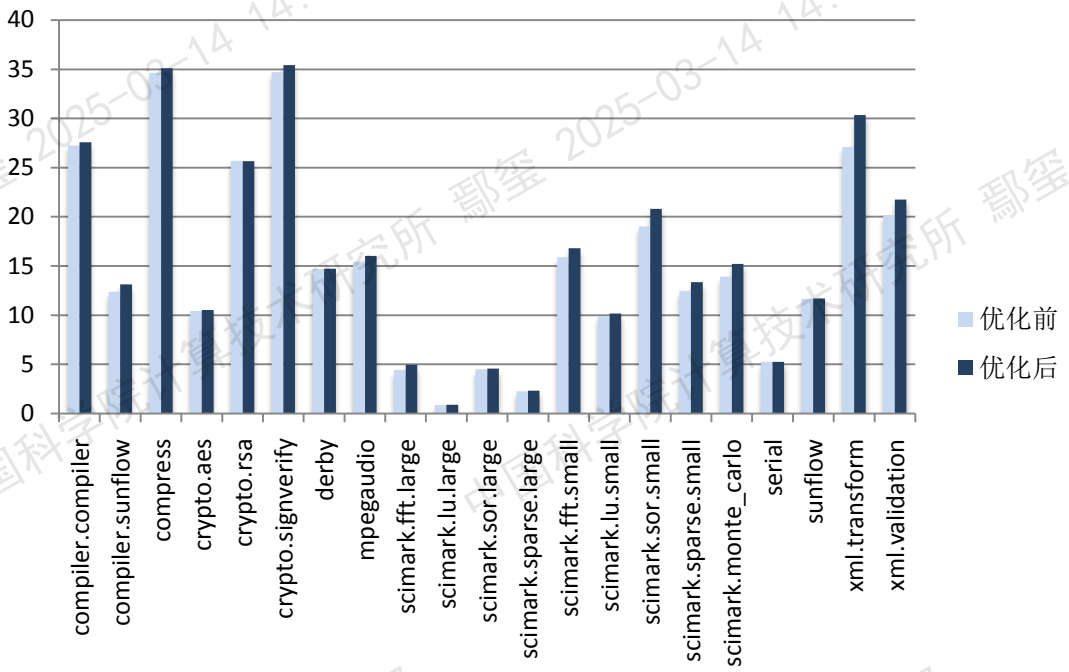

图 5-5 运行时类库热点类优化效果测试

本文提出的热点类优化依赖对目标应用类库使用频度的度量，针对性较强，对于类库方法使用频度分布不均匀的用例优化效果才显著。另外，对热点类的判定是相对的概念，实际应用场景中存在更多潜在的热点类，需要在平台相关性、实现开销与性能之间综合考虑，选择投入优化的深度。

5.4 本章小结

本章以龙芯软硬件平台为测试环境，对本文提出的 64 位立即数加载优化和运行时类库优化方案进行了功能验证和效果测试。测试结果表明，64 位立即数加载优化能够有效提高即时编译器编译后代码质量，减少编译后指令条数，提升执行效率。以 SpecJVM 2008 为测试用例，分值提升最高达到 10.37%，平均测试分值提升 5%。针对运行时类库的优化，利用 JNI 调用和 Intrinsics 机制能够将热

点类库方法本地化，提升执行效率。以 SpecJVM 2008 为测试用例，部分项目性能提升显著，分值提升最高达到 11.93%，平均测试分值提升 4.07%。

中国科学院计算技术研究所 鄢銮 2025-03-14 14:47:37

中国科学院计算技术研究所 鄢銎 2025-03-14 14:47:37

第六章　　总结与展望

在全球云计算和移动互联网的产业环境下，Java 应用因其良好的的通用性、平台无关性和安全性而广泛应用于各个领域。随着用户应用程序规模的增大，应用需求和复杂度的增加，对 Java 应用的执行效率日益提高。Java 虚拟机作为 Java 程序执行的基础平台，其性能对 Java 应用执行效率影响显著。本文结合 MIPS 体系结构特性，针对现代虚拟机即时编译器执行引擎与运行时类库两部分模块分别提出性能优化方案。

6.1 本文工作总结

本文主要工作和创新之处如下：

调研了当前 Java 虚拟机的结构特点及各个模块的工作原理，对 JVM 核心执行引擎以运行时支持等各功能模块进行深入剖析，并总结了近年来对解释器、即时编译器以及垃圾回收器的重要优化方法。

针对虚拟机即时编译器模块，结合 MIPS 体系结构特点，提出了 64 位立即数的优化加载方式。通过在虚拟机内存中增设立即数区，本地代码入口地址，将方法调用和公共例程调用的入口地址的立即数加载转化为访存指令，从而节省指令条数，提升执行效率。

针对运行时类库模块，以 SpecJVM 2008 为典型测试用例，对类库方法使用频度进行度量。度量结果发现，一些典型应用场景下，类库中方法的使用频度分布不均，存在少量高频度执行的热点方法。

针对运行时类库中高频调用的热点方法，提出利用 Java 虚拟机提供的 JNI 调用和 Intrinsics 机制进行改写。针对大型方法通过 JNI 机制将目标方法本地化，针对小型方法通过 Intrinsics 机制将目标方法可执行代码提前内置于虚拟机中，两种方法均能提高翻译后代码执行效率。

针对本文提出的虚拟机优化设计，在龙芯软硬件平台上以 SpecJVM 2008 为测试用例进行了验证与测试，数据表明立即数加载优化节省了 3%指令条数， 带来 5%平均性能提升。针对 SpecJVM 2008 测试用例的热点类投入运行时类库优化，平均带来 4.07%性能提升。

本文部分研究工作和优化成果，已经集成于最新龙芯 JDK 产品中。

6.2 未来研究方向

本文研究过程中，仍有不足之处，体现在如下两方面：

首先，本文提出的立即数加载优化方案，由于全局立即数表容量有限，目标应用翻译方法过多，会导致立即数表填满，虚拟机将退回解释模式继续执行。由于解释模式执行效率较低，长时间运行反而会降低系统性能。

其次，本文对虚拟机运行时类库的优化，仅以 SpecJVM 2008 中的部分典型用例进行了方法使用频度的度量，并仅对最高频的个别方法进行了优化，无论优化的广度和深度都有进一步深入挖掘的空间。

针对上述问题，结合已有研究成果，在如下三个方面值得进一步研究：

第一，即时编译器部分，建立全局立即数表的表项回收机制。该方案的思路是记录立即数表表项的使用频度，将近期不使用的表项进行回收以容纳新的立即数，这种方案能够使立即数表容纳更多方法的入口地址，提高立即数表利用率，从而避免出现立即数表填满，退回解释模式执行的情况。

第二，通过增设特殊硬件指令，突破访存指令只能通过 16 位偏移寻址的限制。借助龙芯自主软硬件平台的优势，可以通过这一方案彻底解决立即数表的容量限制问题，同时访存性能优化研究也对立即数加载性能有重要意义。

第三，运行时类库优化部分，本文以 SpecJVM 2008 为优化案例提出的优化方法仅在一定场景下有适用性。实际场景中，应选择具体的应用作为优化目标，对类库使用频度进行度量。例如可对 tomcat 等中间件进行度量，对高频执行的方法进行本地化或内置于虚拟机中，从而优化系统启动时间。

参考文献

[1] James Gosling, Bill Joy, Guy L.Steele, Java Language Specification[S], Addison-Wesley Longman Publishing Co., Inc. Boston, MA, 1996.

[2] Tim Lindholm, Frank Yellin, Java Virtual Machine Specification[S], Addison-Wesley Longman Publishing Co., Inc. Boston, MA, 1999.

[3] Goldberg, A specification of Java loading and bytecode verification[C]. In ACM Conference on Computer and Communications Security, 1998. pp. 49–58.

[4] Sun Microsystems, Java HotSpot Virtual Machine, A Technical White Paper.

[5] Sun Microsystems, Memory Management in the Java HotSpot Virtual Machine, April, 2006

[6] (英) Dominic Sweetman 著，李鹏，鲍峥，石洋译，MIPS 体系结构透视[M]，机械工业出版社，2008

[7] Wu Hui, Implementation and Optimization of Loongson 64-bit Java Virtual Machine, 2013

[8] 李森，龙芯平台 64 位 JAVA 虚拟机 Server 版的实现与优化，2013

[9] James, E.Smith, Ravi Nair, Virtual Machines: Versatile Platforms for Systems and Processes，安虹，张昱，吴俊敏译，机械工业出版社，2009

[10] 周志明，深入理解 Java 虚拟机，机械工业出版社，2011

[11] John Aycock, A Brief History of Just-In-Time[C], ACM Computing Surveys, June 2003.pp.97-113.

[12] Anderson Faustino da Silva, Vitor Santos Costa, Our Experiences with Optimizations in Sun’s Java Just-In-Time Compilers Journal of Universal Computer Science, vol. 12, no. 7, 2006.

[13] Adl-Tabatabai A-R., Cierniak M, Lueh G-Y, Parikh V, Stichnoth J, Fast, effective code generation in a just-in-time Java compiler[C], Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation. ACM Press, New York, 1998.

[14] Thomas Kotzmann, Christian wimmer, Hanspeter Mossenbock, Thomas Rodriguez, Kenneth Russell, David Cox, Design of the Java HotSpot™ Client

Compiler for Java 6[C], ACM Transactions on Architecture and Code Optimization, Vol.5, May 2008.

[15] Michael Paleczny, Christopher Vick, Cliff Click, The Java HotSpot™ Server Compiler, Proceedings of the Java™ Virtual Machine Research and Technology Symposium, California USA, April 23–24, 2001

[16] BJones, R. Lins, Garbage Collection: Algorithms for Automatic Dynamic Memory Management[C], New York, 1996.

[17] Venstermans K, Eeckhout L, Bosschere K D. 64-bit versus 32-bit Virtual Machines for Java[J]. Software Practice & Experience, 2006, 36(1):1–26.

[18] Pramod Ramarao Improving 64-bit Java performance using Compressed References CASCON 2008

[19] Qi Ao, Guojie Jin, Songsong Cai, Shuaichen, Optimizing Memory Access with Fast Address Computation on a MIPS Architecture, IEEE NAS 2014.

[20] MIPS technologes. Inc. Charles Price MIPS IV Instruction Set, September, 1995

[21] Urs Holzle, Craig Chambers, David Ungar. Optimizing Dynamically-Typed Object-Oriented Languages with Polymorphic Inline Caches[C]. In: ECOOP'91 European Conference on Object-Oriented Programming, vol.512, Lecture Notes in Computer Science

[22] Agesen D D A O. Inlining of Virtual Methods[J]. Lecture Notes in Computer Science, 1999.

[23] Sarimbekov A, A S, Sarimbekov A, et al. Complete and Platform-Independent Calling Context Profiling for the Java Virtual Machine[J]. Electronic Notes in Theoretical Computer Science, 2011, (1):61-74.

[24] S. Liang and G. Bracha, Dynamic class loading in the Java™ virtual machine[C]. Proc.Conf. on Object-Oriented Programming, Systems, Lanages, and Applications, ACM Press, 1988, pp.36-44.

[25] Sheng Liang, The Java Native Interface: Programmer's Guide and Specification [S], 1999

[26] OpenJDK 6 [EB/OL]. http://openjdk.java.net/projects/jdk6/

[27] OpenJDK 8 [EB/OL]. http://openjdk.java.net/projects/jdk8/

致 谢

在进行本文研究工作期间，本人得到了很多老师、同学与亲友的关怀与帮助，也让本人在整个研究生阶段的学习和工作受益匪浅，并充满热情与乐趣。

首先感谢我在中科院计算所的导师章立生老师，在我攻读硕士学位期间在生活与学业上都提供了无私的指导，帮助我顺利完成从本科生到研究生的过渡，使我快速融入实验室氛围，为我在知识和技能上的积累营造了良好的环境。

感谢龙芯中科技术有限公司总裁胡伟武老师，三年以来在思想上，科研上都为我们树立了标杆。胡老师常常教导我们“君子先识器而后文章”，他崇高的道德素养和爱国主义情怀一直激励着我们。

感谢中科院计算所微处理器研究中心的王剑老师和章隆兵老师。两位老师在我就读期间一直负责学生工作，在本文的完成过程中也为我提出了宝贵意见，两位老师深厚的学术素养与严谨的学术态度是我学习的榜样。

感谢龙芯中科技术有限公司研发中心的靳国杰老师，在我参与实验室研发项目过程中在知识上的传授以及技能上的培养，作为我技术成长道路上的重要引路人，靳老师的工作态度与方法，一直激励着我不断学习和进步。

感谢龙芯系统研发部 Java 组的廖彬、傅杰两位师兄，从我进组之初就在工作上给予我关怀。生活中，两位师兄也给我很多照顾，帮我解决了生活中的很多困难，也让我繁忙的科研生活中感到温暖。

感谢龙芯系统研发部 Java 组的其他成员，王振华、毕研儒、王雪倩、任博文，在科研工作中团结友爱，互相帮助，让我感觉到很强的集体凝聚力，一起工作的两年时光令人难忘。

感谢 2015 届微处理中心的全体同学，三年来我们一起奋斗拼搏，共同成长，相互鼓励，相互扶持。研究生阶段能与你们一起科研学习，我感到非常荣幸。

感谢我在研究生就读期间的两位室友，吴鹏、吴俊。三年来我们相处融洽，志同道合，留下了很多难忘而珍贵的回忆。

感谢我的家人和朋友，感谢你们多年以来一直陪伴我，在成长和求学的道路上给我支持，给我力量。

中国科学院计算技术研究所 鄢玺 2025-03-14 17:37

作者简介

姓名：王一光　　性别：男　　出生日期：1990.8.22　　籍贯：陕西西安

2012.9 – 2015.7　　中科院计算所计算机系统机构专业　　硕士
2008.9 – 2012.7　　西北工业大学计算机科学与技术专业　　学士

中国科学院计算技术研究所 鄢玺 2025-03-14 14:47:37